

Robust Elections and More: Fast MPC in the Preprocessing Model

Charanjit S. Jutla¹, Nathan Manohar¹, and Arnab Roy²

¹ IBM T.J. Watson Research Center

² Mysten Labs

Abstract. In this paper, we present an MPC protocol in the preprocessing model with essentially the same concrete online communication and rounds as the state-of-the-art MPC protocols such as online-BGW (with precomputed Beaver tuples) for $t < n/3$ malicious corruptions. However, our protocol additionally guarantees robustness and correctness against up to $t < n/2$ malicious corruptions while the privacy threshold remains at $n/3$. This is particularly useful in settings (e.g. commodity/stock market auctions, national elections) where it is paramount that the correct outcome is certified, while maintaining the best possible fast-track online speed. In addition, this honest-majority correctness allows us to use optimistic Berlekamp-Welch decoding in contrast to BGW. Moreover, just like online-BGW, our protocol is responsive until a final attestation phase.

We also give a complementary verifiable input-sharing scheme for the multi-client distributed-server setting which satisfies both robustness and correctness against up to $t < n/2$ malicious servers. This is accomplished by having the servers first run a preprocessing phase that does not involve the clients. The novelty of this input-sharing scheme is that a client only interacts for one round, and hence need not be online, which, again, is highly desirable in applications such as elections/auctions.

We prove our results in the universally-composable model with statistical security against static corruptions. Our protocol is achieved by combining global authenticators of SPDZ with an augmented Reed-Solomon code in a novel manner. This augmented code enables honest-majority decoding of degree $n/2$ Reed-Solomon codes. Our particular augmentation (often referred to as robust sharing) has the additional property that the preprocessing phase can generate this augmented sharing with a factor n speedup over prior information-theoretic robust sharing schemes.

1 Introduction

Secure multi-party computation (MPC) is a fundamental cryptographic primitive that allows a set of mutually distrusting parties $P_1, \dots, P_n$, each possessing an input x_i , to compute a function $f(x_1, \dots, x_n)$ on their joint inputs without

revealing anything other than the output of the function. Such protocols are said to be *maliciously secure* if this holds even against an adversary that corrupts t of the parties and has them deviate arbitrarily from the protocol. In this work, we focus on maliciously secure protocols in the honest majority setting, where $t < n/2$. The honest majority setting is of particular interest because in this setting, it is possible to achieve *guaranteed output delivery* (GOD), which means that the honest parties receive the output of the computation regardless of the behavior of the t corrupted parties. This is particularly important in settings where it is crucial that the protocol does not fail. In the honest majority setting, security can be achieved against a computationally unbounded adversary (perfect security in the case $t < n/3$ [BGW88, BTH08] and statistical security in the case $t < n/2$ assuming a necessary [RB89, GSZ20] broadcast channel.

Over the years, MPC protocols have been split into an *offline stage*, also known as a preprocessing stage, and a highly efficient *online stage* [Bea91a]. The offline stage generates correlated randomness for the parties before the parties have the inputs to the protocol. When the inputs are set, the parties can run a much faster online stage, capitalizing on the correlated randomness. The online phase usually enjoys further important properties such as information-theoretic security, a potentially asynchronous network setting, and the information-theoretic protocol also usually implies shorter messages. In this work, we focus on MPC protocols in this preprocessing model and focus on the efficiency of the online phase of MPC protocols with perfect/statistical security against a computationally unbounded adversary.

A fundamental MPC protocol is the GMW protocol [GMW87], which showed how to compute any function securely against a semi-honest adversary that corrupts up to $n - 1$ parties. GMW can be implemented using cryptographic protocols such as OT or using precomputed *Beaver multiplication triples* [Bea91a]. When using precomputed Beaver tuples, the online communication complexity is as low as *two* field elements per party per multiplication gate. The round complexity per multiplicative depth is two as each party just sends their shares to one designated party, who then performs the opening and sends back the result to all parties. Later protocols such as SPDZ [DPSZ12] (following [BDOZ11]) achieved essentially the same concrete online communication complexity while making the protocol maliciously secure.

One downside of the above GMW-style protocols ([GMW87, BDOZ11, DPSZ12]) is that they do not provide any fault-tolerance or robustness against malicious behavior. In 1988, BGW [BOGW88] showed that for up to $n/3$ corruptions there is an information theoretic maliciously secure private *and* robust protocol. Again using pre-computed Beaver tuples, the online phase required n field elements of communication per party per multiplication gate (although in one round). It was eventually optimized in [DN07] to *two* elements of communication by each party per multiplication gate (in two rounds, with all gates at the same depth handled together, assuming each depth has at least n gates). This round complexity then matches the online-GMW round complexity. We consider these fundamental protocols as gold standards in concrete online communication and rounds,

and our aim in this work is to realize further gains in security/robustness while maintaining this gold standard of concrete online communication and rounds.

In the $t < n/3$ corruption setting, the state of affairs is quite satisfactory. In the aforementioned model, it is possible to achieve perfectly secure MPC, where the online phase requires $O(n|C|)$ total communication (where $|C|$ denotes the size of the circuit to be computed measured in the number of multiplication gates) and $O(D)$ rounds (where D is the depth of the circuit C) [BOGW88, BTH08]. Note that $O(n|C|)$ total communication means that each party’s communication does not increase as the number of parties increases, which is desirable for the scalability of the protocol. Moreover, as previously mentioned, the constants hidden in the big-O notation for communication and round complexity can be 2. Furthermore, the protocol is *responsive* [PS17], which informally means that the protocol can be executed at the speed of the actual network delay in each round rather than requiring all parties to wait for a (potentially very loose) upper bound Δ on the network delay before proceeding to the next round.

In contrast, in the $t < n/2$ corruption setting, the state of affairs is much less satisfactory. The protocols with the most efficient online communication complexities are those of [GSZ20] and [EF23], which are able to achieve $O(n|C|)$ total online communication, asymptotically matching the results of the $t < n/3$ corruption setting as far as cost per gate is concerned. However, the protocol of [GSZ20] has a round complexity of $\Omega(D+n^2)$ due to its reliance on the dispute control technique, which is a major drawback when the number of parties is large. Moreover, the actual communication complexity is $O(n \cdot \phi \cdot |C| + n^3 \cdot \lambda^2 + n^6 \cdot \lambda)$ bits plus $O(n^3 \cdot \lambda)$ bits on the assumed broadcast channel, where ϕ is the size of a field element and λ is the security parameter. While the cost per gate is optimized to about 5.5 field elements per multiplication gate, the overhead of $O(n^3 \cdot \lambda)$ bits on the assumed broadcast channel and the $O(n^6 \cdot \lambda)$ bits of communication make this inefficient for real-world applications. This advance, however, makes offline-online MPC realistic by supporting offline preprocessing which is crucial to implement the online phase securely. The offline-online protocol of [EF23] has an online phase consisting of $O(n|C|)$ total communication and $O(D)$ rounds, asymptotically matching the results of the $t < n/3$ corruption setting. Unfortunately, the protocol of [EF23] comes with several drawbacks, as we highlight next.

- **Hidden Constants:** Although the online communication and round complexity of [EF23] asymptotically matches the results for the $t < n/3$ case, the concrete costs are significantly higher. The constant hidden in the big-O notation in this protocol is 7 as opposed to 2 in the state-of-the-art protocols for the $t < n/3$ case. Furthermore, these asymptotic communication complexities are measured in terms of field elements sent by the parties, but the sizes of the fields greatly differ in these two settings. In the $t < n/3$ case [BOGW88, BTH08], the field elements communicated come from some field $\mathbb{F}$, where $|\mathbb{F}| > n$ (required for valid Shamir sharings to be given out to n parties over $\mathbb{F}$). However, in the $t < n/2$ case, the protocol of [EF23] requires $|\mathbb{F}| > n^2 \cdot 2^\lambda$, where λ is the statistical security parameter. Thus, the concrete

communication costs of [EF23] are at least $\frac{7}{2}(2 + \lambda/\log n)$ times larger than those of [BOGW88, BTH08].

- **Circuit Width:** A further drawback of [EF23] is that it requires the width of C to be $\Omega(n^2)$, a factor of n more than the $\Omega(n)$ required for the $t < n/3$ case (which is necessary in order to batch the opening of shares corresponding to gates at the same depth [DN07]). The reason for this additional requirement comes from the fact that in [EF23], instead of using a global MAC key as in SPDZ [DPSZ12] for authentication and correctness, they use robust secret sharing as in [DMO⁺11, BPRW16]. However, they manage to reduce the communication complexity by batching the robust secret sharing decoding. For this batching to work, they critically require $\Omega(n^2)$ width for the circuit as well as a commit and open phase for every depth. If this width requirement on the circuit is removed, then [EF23] incurs an additional $O(n^3D)$ communication, whereas in the $t < n/3$ setting, BGW-style protocols [BGW88, BTH08] incur only an additional $O(n^2D)$ communication. This difference can be quite large in practice, especially when the number of parties n is large.
- **Non-responsive Protocol:** Finally, the protocol of [EF23] is non-responsive, meaning that it must be run over a partially synchronous network with an upper bound Δ on the network delay, and all parties must wait Δ time in each round, even if the actual network delay is significantly less than Δ . This is in contrast to the $t < n/3$ case, where the protocol is responsive, meaning that parties that have received messages from all other parties for a round can proceed to the next round without waiting a fixed amount of time Δ . Since the actual network delay is often significantly less than the upper bound Δ , responsive protocols can run and finish in a fraction of the total time compared to non-responsive protocols.

Thus, there still remains a significant gap in the complexity of the online phase of MPC in the preprocessing model between the $t < n/3$ and $t < n/2$ settings, and further closing this gap was listed as an open problem in [EF23].

Let's take a step back and examine the properties provided by MPC protocols in the above setting. Informally, the protocols in this setting achieve the following guarantees.

- **Privacy:** $< t$ malicious parties learn nothing other than the output.
- **Robustness/GOD:** All honest parties correctly compute the output even if $< t$ of the parties are malicious.
- **Correctness:** $< t$ malicious parties cannot cause an honest server to output incorrectly, i.e. to some $y' \neq f(x_1, \dots, x_n)$.

Prior works assumed the same corruption threshold t for all three guarantees. However, it is possible for a protocol to satisfy privacy, robustness, and correctness for different corruption thresholds t_p , t_r , t_c , respectively. It is clear that correctness is implied by robustness for the same threshold t , and, therefore, $t_c \geq t_r$. However, it is possible that a protocol may satisfy privacy for some corruption threshold t_p and, additionally, satisfy robustness/correctness

for larger thresholds t_r, t_c . This is particularly relevant in settings, such as elections/auctions, where honest (majority) parties accepting an *incorrect* output (such as the winner of an election) can be catastrophic. In this work, we focus on the following question.

Can we construct an MPC protocol in the preprocessing model that satisfies privacy against $t < n/3$ malicious corruptions, but robustness and correctness against a larger number $t < n/2$ malicious corruptions while still maintaining high efficiency in the online phase?

By maintaining high efficiency, we mean that the online phase of the MPC protocol has almost the same communication/round complexity as the online phase of the most efficient protocols in the $t < n/3$ setting [BGW88, BTH08] (i.e. up to a marginal overhead in cost –see Table 1 for a look-ahead). Of course, as stated above, we demand extra robustness and correctness against a larger number $t < n/2$ of malicious corruptions. An immediate advantage of this protocol over online-BGW is that it can employ an optimistic errors-and-erasures Berlekamp-Welch decoding (see Section 1.4 below).

1.1 Our Contribution

In this work, we answer the above question affirmatively by constructing an MPC protocol in the preprocessing model (with circuit-independent preprocessing) that satisfies privacy against $t < n/3$ malicious corruptions and robustness and correctness against $t < n/2$ malicious corruptions. While boosting the correctness threshold to $n/2$ requires minimal overhead similar to a SPDZ like final aggregate check, the robustness against dishonest minority requires a re-run of the online phase using a more communication expensive robust code. But, importantly this re-run is only needed if honest parties detect that more than $n/3$ parties were malicious and had derailed the computation. In this respect, it is worth mentioning that such protocols are called fast-tracked [GRR98], in the sense that if everything is fine (in our case less than $n/3$ corrupt), the online phase of our protocol requires a total communication of $2n|C|$ elements of a small base field $\mathbb{F}$ of size $> n$, and an additional communication of $\text{poly}(n)$ elements of a larger extension field $\mathbb{E}$ (of size $> |C| \cdot n^2 \cdot 2^\lambda$). The round complexity of our (responsive) protocol is $2D + O(1)$ rounds. Moreover, the circuit width requirement for our protocol is $\Omega(n)$, matching the width requirement to obtain this communication complexity in the $t < n/3$ setting.

Our protocol is written in the client-server setting, where we view the n servers as the parties executing the protocol, and the clients as providing inputs to the protocol via an input sharing phase that takes place as part of the preprocessing after the generation of some correlated randomness shared amongst the n servers.

We provide a complementary information-theoretic verifiable input sharing scheme that has robustness against $t < n/2$ corruptions, so that we ensure that a client’s input cannot be incorrectly misconstrued by the servers (or outright

Protocol	Corruption Thresholds	Comm. (* $\log \mathbb{F} $)	Field Size	Circuit Width	Rounds	Responsive Rounds
BGW-style [BOGW88, BTH08]	$(n/3, n/3, n/3)$	$2n C $	$> n$	$\Omega(n)$	$2D$	✓
SPDZ [DPSZ12]	$(n, 1, n)$	$2n C + \text{poly}(n)$	$> n^\dagger$	$\Omega(1)$	$2D + O(1)$	✓*
EF23 [EF23]	$(n/2, n/2, n/2)$	$7n C $	$> n^2 \cdot 2^\lambda$	$\Omega(n^2)$	$7D$	✗
This work	$(n/3, n/2, n/2)$	$2n C (\text{fast-track}) + 1 \text{ Byz}^{**} + \text{poly}(n)$	$> n^\dagger$	$\Omega(n)$	$2D + O(1)$	✓*

Table 1: Comparison of the online phase of information-theoretic MPC protocols with $O(D)$ rounds in the preprocessing model. The corruption thresholds are of the form (t_p, t_r, t_c) , the thresholds for privacy, robustness, and correctness, respectively. **Byz** denotes the communication of an honest-majority Byzantine consensus which can be run in expected constant rounds assuming the servers have certified public-keys [KK06]. The double asterisk on **Byz** signifies that if no party was erroneous then the Byzantine consensus can be stopped short asynchronously (see last step of Protocol $\Pi_{\text{ONLINE-X-SFE}[\mathbb{F}, f, n, m]}$). In other words, **Byz** has an impact on total communication only if more than $n/3$ parties acted maliciously. ✓* means the protocol is responsive until the attestation phase at the end (the final $O(1)$ rounds).

† Except for the $\text{poly}(n)$ communication of the attestation phase, which requires a larger field, or if more than $n/3$ parties were corrupt and had cause attestation to fail, in which case the online-BGW is re-run with robust codewords.

disqualified) as long as there is an honest majority. This is crucial since the ability to change a client’s input during the input sharing phase would also affect the final output, and, therefore, robustness/correctness of the protocol. Moreover, the input sharing phase is a simple one-round interaction, so the clients’ persistent presence is not required. In addition, the client is not required to be part of the preprocessing.

We also give a new verifiable secret sharing (VSS) scheme, inspired by [DMO⁺11], and sometimes called *robust sharing* [EF23], that is used as a crucial building block in our protocol. While the size of the preprocessing material used by this robust sharing protocol is essentially the same as that of [EF23], our robust sharing scheme shaves off a factor n from the communication complexity of the offline phase for this sharing as compared to [EF23]. This effectively shaves off an n factor from the overall complexity of the offline phase.

1.2 Further Motivating Examples

Suppose there is a sealed-bid auction scheduled for a specific date and time, and we wish to run an MPC protocol amongst n servers to determine the winning bid(s). In this scenario, the servers have a comparatively long period of time

to generate the preprocessing material in the offline phase, and clients (bidders) have a fixed period of time (say an hour or so) in which they can submit their bids (via input sharing to the servers). However, once the bidding ends, we would like the online phase of the protocol to be fast, so that the results can be declared quickly (within seconds). In many auctions, e.g. the stock markets such as New York Stock Exchange (NYSE), the opening auction has to be completed as fast as possible [NYS]. Note that the outcome of the auction (the circuit to be computed) need not be simply computing the highest bidder, but could be some more complicated functionality, for example the clearing-price double-auction (CPDA). In this scenario, privacy is important since a bidder's bid should remain private (the bids/asks that did not win/clear). However, robustness and correctness are of utmost importance since, otherwise, an incorrect winner could be declared (maliciously) or the auction could fail and have to be rerun. A similar situation arises in elections and voting, where such a malicious incorrect outcome can be catastrophic. In general, an adversary would be more interested/motivated in affecting robustness to install its preferred bidder as the winner than it would be to learn a particular bidder's bid. Thus, we view our protocol with increased robustness/correctness thresholds as applicable in scenarios such as this, where not certifying an incorrect output is paramount as is receiving the result in a timely manner. We emphasize that sealed-bid CPDA auctions are one of the few instances of real-world deployment of MPC (see [BCD⁺09]). Other examples of multi-client distributed-server setting include large-scale elections, and MPC on permissioned-blockchains. See Appendix D for comparison of speed of our protocol (and its preprocessing) vs running an honest-majority MPC protocol.

1.3 Technical Overview

BGW-style MPC protocols operate with the inputs/intermediate values secret shared amongst the parties. When using Beaver tuples generated in the preprocessing phase, the online phase operates by consuming a Beaver tuple per multiplication gate and having the parties open blinded shares. When $t < n/3$, malicious behavior can easily be error corrected, *guaranteeing* that honest parties receive the correct openings and eventually the correct output(s). However, in the $t < n/2$ case, malicious behavior is not easily error corrected. There are two main approaches to handle this honest-majority setting efficiently (in the preprocessing model):

1. One approach is to use an $n/2$ -robust secret sharing scheme for all the underlying shares of the protocol [EF23]. There are two drawbacks to this approach: (a) The error-correction must happen in each round (i.e. per level of the circuit), which as we note next can be relatively expensive as well as non-responsive. (b) The shares of the $n/2$ -robust secret sharing scheme are significantly larger due to the usage of large fields, which is necessitated because in order to determine which shares were honestly given and which are malicious (in order to eliminate the malicious ones to properly reconstruct), we need a method of verifying these shares. This can be done, for

example, by having each party possess secret evaluation points unknown to the other parties. For statistical security, the field must be large enough that these points are not easily guessed. Further, this requires a commit and open scheme which then requires synchronous rounds which are not responsive.

2. The other approach uses the observation that (most) semi-honest MPC protocols only have “additive” attacks [GIP⁺, GIP⁺15b] and tries to compile them into maliciously-secure schemes using MACs (as in SPDZ), see e.g. [LN17, CGH⁺18]. These can indeed be very fast, satisfying our speed criteria, essentially by doing an aggregate correctness test at the end. However, in the honest-majority setting, these protocols only achieve security-with-abort since they use a *non-robust* secret sharing scheme. So, these protocols fail our robustness criteria. One cannot employ robust secret sharing, as the opening of $n/2$ -robust secret sharing schemes requires commit and open and larger share sizes, and we are then back in case 1.

Our approach is to employ both these ideas in a novel way to get fast MPC that has $n/3$ privacy threshold, but honest majority correctness and robustness. Since we are focused on the online stage, we will employ pre-computed Beaver triples as well as authenticated Beaver triples. The main idea is to use an $n/3$ -privacy threshold secret sharing code for the basic Beaver triples, and an $n/2$ -robust code for the authenticated tuples. During the execution of the protocol, the authenticators are updated but not opened. Then, at the end of the protocol, there is an attestation phase, where these authenticators are aggregated and then opened, attesting to the entire computation with a low communication/round overhead. This is similar to the approach employed by SPDZ; however, since we are using sharings at two different thresholds, getting the underlying shares and the authenticated shares to work together is a challenge. At a high level, the “additive only” adversary hypothesis fails now since if more than $n/3$ servers are corrupt (though still less than $n/2$), privacy is lost; but it was the privacy of the semi-honest protocol that allowed one to conclude that the only possible attacks are “additive only” attacks. To be precise, [GIP⁺15b, Theorem 5.2] shows that if a “linear” MPC scheme is t -private, then it is t -secure up to additive attacks.

So, how do we prove that our scheme achieves correctness? The first observation is that while the privacy of the clients’ inputs are lost (as these were secret shared using an $n/3$ -threshold scheme), the authentication key is still private (as the authenticators were secret shared using an $n/2$ -threshold scheme), and maybe this allows us to show that attacks are “additive only”, with additions independent of the authentication key. This is only partially true³, and counterexamples can be given that iterative MACs computed by parties can collectively have an additive error that is not independent of the key. But, what we can show is that if it is guaranteed that up to some depth d of the circuit the computation was correct, then in the next depth $d + 1$ the errors introduced by the adversary are “additive only”. That is, they are independent of the authentication key (the

³See [GIP15a] for another example of a semi-honest secure scheme where the “additive only” adversary hypothesis fails, yet the overall strategy works out – this technique is unrelated to our work.

same cannot be said for the errors introduced at depth $d + 2$ etc.). We leverage this observation to give an inductive proof of correctness using a single algebraic aggregation done during attestation (employing the Schwartz-Zippel lemma). See Lemma 7.2 and the accompanying proof for more details.

The above provides a roadmap for enhancing the correctness threshold to $n/2$. However, once this is done, it turns out that the robustness threshold can easily be upgraded to $n/2$ as well. This follows from the fact that if $t < n/3$, the protocol is already robust and so the output will be correctly computed by the honest parties. Thus, if the parties are unable to come to a consensus on the output (no value was in the majority), then it must be the case that $t \geq n/3$. Since the parties already have robust-code secret-shares of Beaver multiple tuples (one for each gate), they just re-run the online-BGW with these robust Beaver multiplication -tuples. The crucial point is that this only happens in the mostly unlikely scenario that more than $n/3$ parties were maliciously corrupt.

1.4 Immediate Advantage Over BGW

We now illustrate a surprising advantage of our enhanced-correctness protocol over BGW. We will assume that both BGW and our protocol use the Berlekamp-Welch erasure-and-errors decoder which can decode correctly if $e + 2s \leq 2n/3$, where s is the number of erasures and e is the number of errors.

While it is well-known that the BGW protocol can be tricked into outputting an incorrect result when $e + 2s > 2n/3$ (whereas our protocol will not attest such an incorrect result as long as $e + s \leq n/2$), we now show that using denial-of-service attacks, just $n/4 + 1$ malicious parties can cause BGW to output an incorrect result.

Suppose $n/4 + 1$ parties, say set E , are corrupt and they try to convince/hack some other $n/12$ parties to join them so that their quorum is above $n/3$, but fail to do so. The parties E may then instead launch a denial-of-service (DOS) attack on some, say $n/6$, parties S . Thus, now $2e + s = 2 * |E| + |S| = n/2 + 2 * n/6 > 2n/3$, and as described next, E can judiciously force BGW to output an incorrect result. This is well-known in coding theory, but with $n/6$ erasures the code is now truncated, effectively of output length $5n/6$ (with rank still $n/3 + 1$). The minimum distance is now $5n/6 - n/3 = n/2$. Thus, error-correction is only guaranteed upto $n/4$ errors. In particular consider a codeword $\vec{c}$ (of this truncated code) of hamming-weight $n/2$, which is non-zero in the $n/4 + 1$ slots corresponding to E , and also non-zero in some other $n/4 - 1$ slots (say, corresponding to some honest parties G). It is zero in the remaining $5n/6 - n/4$ slots. Parties E send the original message but with $\vec{c}|_E$ added in. The Berlekamp-Welch erasure-and-error decoder will decode without failing and declare the G parties to be erroneous. Our enhanced-BGW protocol will however reject the attestation because $e + s = n/4 + 1 + n/6 < n/2$ (unless the parties fortuitously managed to compute correctly).

For this reason, a BGW implementation may choose to differentiate between erasures that correspond to non-responding parties and those that explicitly respond that they have suffered failure (e.g. due to a previous round decoding error). To be precise, let e be the number of parties that deviate from protocol

and add some error to their outgoing message. Let s_{dos} be the number of parties that fail to respond, and let s_{fail} be the number of parties that respond with a failure indicator. Now, suppose the erasure-and-error decoder passes decoding and computes that f parties had error; it is easy to show that $f \leq e$. The BGW-decoder may follow the policy that if $f > 0$ and $f + s_{\text{dos}} > n/3$ then it declares failure, essentially concluding that at least one party is erroneous and potentially $n/3 + 1$ (i.e. $f + s_{\text{dos}}$) parties are controlled by the Adversary. This then prevents the incorrect output example scenario above. However now robustness is no more guaranteed under $2e + s_{\text{dos}} + s_{\text{fail}} \leq 2n/3$. This is why the enhanced-BGW protocol is better as robustness is guaranteed under $2e + s_{\text{dos}} + s_{\text{fail}} \leq 2n/3$ and $e + s_{\text{dos}} < n/2$ and correctness is guaranteed even if $2e + s_{\text{dos}} + s_{\text{fail}} > 2n/3$ as long as $e < n/2$.

1.5 Improved Preprocessing

Our protocol utilizes a new verifiable secret sharing (VSS) scheme inspired by [DMO⁺11]. A description of this sharing scheme is given in Fig. 2.

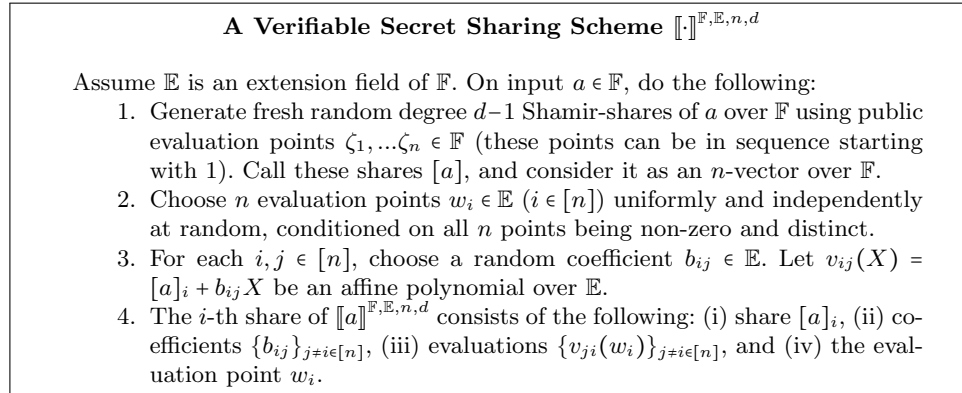


Fig. 1

At a high level, this sharing scheme works by taking a Shamir sharing $[a]$ of a and using the affine check $v_{ij}(X)$ evaluated at the secret evaluation point w_j to authenticate the i -th share $[a]_i$ to the j -th party. Since w_j is secret, the i -th party is unable to produce a different $v_{ij}^*(X) \neq v_{ij}(X)$ that is consistent with the j -th party's stored evaluation $v_{ij}(w_j)$. Thus, only honest shares will be accepted, allowing for reconstruction of a during opening without error correction. See Section 3 for more details. One can view this authentication check as an example of an information-theoretic MAC using oblivious linear evaluation (OLE) [NP06], a primitive that has seen extensive applications in the context of two-party computation.

Our VSS scheme differs from the one in [DMO⁺11] in the following respect. Instead of using pairwise affine checks $v_{ij}(X)$ and the corresponding evaluation $v_{ij}(w_j)$, [DMO⁺11] uses a single authentication polynomial $v_i(X)$ of degree $d-1$ (with constant term $[a]_i$) to authenticate to all parties, and the j -th party is given the evaluation $v_i(w_j)$ at its secret point w_j . For an adversary corrupting $< d$ parties, it does not have enough evaluations of $v_i(X)$ to recover the constant term $[a]_i$, and so privacy holds. The VSS scheme of [DMO⁺11] is essentially the same as the one used in [EF23]. While the VSS scheme of [DMO⁺11] would have sufficed for our protocol, we use the one in Fig. 2 because it enables a faster preprocessing phase. Although [EF23] does not instantiate their preprocessing phase, to do so they would need to compute the evaluations $v_i(w_j)$, where $v_i(X)$ is a degree $n/2 - 1$ polynomial. In contrast, our authentication checks $v_{ij}(w_j)$ are simply affine checks. In essence, this shaves off an n factor from the overall complexity of the offline phase and so even though the size of our preprocessing material is similar to that of [EF23], it can be generated faster. We discuss instantiating the preprocessing phase in more detail in Supplementary Material C.

1.6 Proof Explanation

There are several prior works on MPC that prove malicious security by first showing that the semi-honest protocol is actually secure up to additive attacks. Then, the protocol is transformed to be fully secure. In our case, the additive only assumption does hold in a simulation model where the simulator is given access to the inputs and Beaver tuples (everything shared at threshold $n/3$) when $t \geq n/3$. So, the additive errors introduced in this case may depend on the inputs, but they are still independent of the MAC key α since α is shared at threshold $n/2$, and we are working in the $t < n/2$ setting.

The challenge is to prove the SPDZ-style attestation at the end is secure. We are in a unique setting where we want robustness as well (at least for $t < n/3$), so we need to use a robust sharing scheme, whereas prior works that only offered security with abort didn't need to use a robust sharing scheme.

The two main comparisons for the proof technique are SPDZ (which is dishonest majority) and [CGH⁺18] (which is honest majority, but not robust). There are two main points that affect the challengingness of the proof (and distinguish our proof from prior works). (1) In the honest-majority setting, the honest parties shares themselves determine the value, so it is not true that the errors introduced in the MAC shares are independent of α . For example, by opening a share to $x + c$ instead of x , the adversary is able to introduce an error of $c \cdot \alpha$ into the MAC check, where c is independent of α . In the dishonest majority case, the honest parties' shares by themselves do not determine the shared value. This fact simplifies the proof in SPDZ. (2) SPDZ-style attestation involves a gate-by-gate aggregation and a single check at the end. In SPDZ, this is done by opening a random x and combining the individual gate checks with powers of x . However, in [CGH⁺18] they use a different random value as the linear combiners for the gate checks. This results in having to open a different random value for every multiplication gate in the circuit, rather than a single value. However, this

greatly simplifies the proof because they can argue that one of the combiners (say x_i) is independent of all other values, and thus, the probability of equality is $1/|\mathbb{F}|$. If they had used a single x for aggregation, this would be false, since the equation on the other side would contain other powers of x .

We have a more involved proof that is able to handle the challenges of the honest-majority setting and use a single x for aggregation. At a high level, we analyze the errors introduced by the adversary in the adversary's own shares and the honest parties' shares separately. The errors introduced in the adversary's shares are independent of α , whereas the errors introduced in the honest parties' shares may depend on α , but, crucially, are of a specific form $c \cdot \alpha$ with c independent of α . We are able to conclude that for the attestation to pass, an equality of the form $W = \alpha \cdot V$ must hold for W, V independent of α . Intuitively, one can think of W as the accumulation of errors from the adversary's shares and V as the accumulation of errors from the honest parties' shares with α factored out (here, we crucially use that the errors introduced in the honest parties' shares are of the form $c \cdot \alpha$). From this, we can conclude that $W = V = 0$ with overwhelming probability. At this point, we are able to apply Schwartz-Zippel to V (an aggregation using x) because while the adversary may have introduced errors into the individual gate checks of the honest parties, the honest parties will honestly aggregate these terms and, therefore, we are guaranteed that V is a polynomial in X . This allows us to conclude that all the individual gate checks are themselves 0, and then, by an inductive argument, we conclude that all errors were in fact 0.

1.7 Technical Overview: A Coding Theory Perspective

Recall that our goal is to obtain an MPC protocol in the pre-processing model with similar online efficiency to the best known protocols with $t < n/3$ such as the online phase of BGW, while achieving robustness/correctness against a larger $t < n/2$ malicious corruptions. All the protocols previously discussed (i.e. SPDZ, BGW, etc.) use linear codes to share secrets amongst the parties. In the case of SPDZ, the linear code is rather simple, in that the secret is additively shared. In more general terms, a secret is said to be **randomly shared** using an $[n, k]$ linear code $\mathbf{G}$, where $\mathbf{G}$ is a *full-ranked* $n \times k$ matrix over $\mathbb{F}$ (or some extension field of $\mathbb{F}$), if the n -shares of a secret $a \in \mathbb{F}$ are $\mathbf{G} \cdot \mathbf{f}$, where $\mathbf{f}$ is a vector randomly and uniformly chosen from $\mathbb{F}^k$, conditioned on $\mathbf{f}_0$ being a . The sharing can also be viewed as a *probabilistic encoding* represented by the diagram $\mathbb{F} \xrightarrow{\$} \mathbb{F}^k \rightarrow \mathbb{F}^n$, where the first map always appends $a \in \mathbb{F}$ with $k - 1$ random elements of $\mathbb{F}$, and the second map is $\mathbf{G}$.

Thus, for SPDZ, or additive sharing $\mathbb{F} \xrightarrow{\$} \mathbb{F}^n \rightarrow \mathbb{F}^n$, the matrix $\mathbf{G}$ (which, for SPDZ, we will call $\mathbf{G}^*$) is a rather simple $n \times n$ matrix (i.e. $k = n$), where the first row is the all 1 vector, and the remaining rows form an $(n - 1) \times n$ matrix, where the first column is the all zero vector, and the remaining $(n - 1) \times (n - 1)$ submatrix is the identity matrix I_{n-1} . Since $\mathbf{G}^*$ is a full-ranked $n \times n$ matrix, this code's minimum distance or minimum weight is one! It is well-known (and

easy to see) that a linear code with minimum distance d can only be uniquely error-corrected for up to $\lfloor \frac{d-1}{2} \rfloor$ errors [vL91, Section 3.2]. Thus, the code used in SPDZ provides no error-correction. However, on the positive side, it provides privacy-protection against $n-1$ parties, as given any of the $n-1$ shares (out of the n), the vector $\mathbf{f}$ is still in a rank one subspace of $\mathbb{F}^k$. Thus, this code provides zero robustness, but privacy against $< n$ corrupted parties.

We call ECC an error-correcting procedure⁴ for $\mathbf{G}$ if given a word $\mathbf{w}$ which is a distance at most $\lfloor \frac{d-1}{2} \rfloor$ from the code, $\text{ECC}(\mathbf{w})$ outputs the (guaranteed) *nearest* codeword. Moreover, if $\mathbf{w}$ is not at a distance at most $\lfloor \frac{d-1}{2} \rfloor$ from the code, $\text{ECC}(\mathbf{w})$ outputs failure⁵. This implies that if a received word has up to $\lfloor \frac{d}{2} \rfloor$ errors, then an attempt to error-correct using ECC is guaranteed to either recover the original message or fail. Thus, we can say that this procedure implies that the code has robustness against $\lfloor \frac{d-1}{2} \rfloor$ errors and *correctness* against $\lfloor \frac{d}{2} \rfloor$ errors (i.e. a marginal gain). In the case of SPDZ, this procedure by itself thus provides no correctness either. However, SPDZ does provide correctness against up to $n-1$ malicious parties by utilizing a separate authentication mechanism, which we describe next.

In SPDZ, the sharing is augmented by randomly sharing a using the original $n \times n$ matrix $\mathbf{G}^*$, as well as randomly sharing $a \cdot \alpha + \beta$ using the same $\mathbf{G}^*$ and also randomly sharing both α and β using the same $\mathbf{G}^*$, where α, β are randomly and uniformly chosen secrets from $\mathbb{F}$ (assume, for simplicity, that $\mathbb{F}$ is large enough to have 2^λ elements, where λ is the security parameter). These additional shares will be used to authenticate a . Observe that a , $a \cdot \alpha + \beta$, α , and β all remain private against up to $n-1$ colluding parties. However, upon opening all four set of shares (say, with the shares of α, β opened⁶ (synchronously) after the first two sets have been opened), even $n-1$ corrupt (and rushing) parties cannot cause the “affine-relation” test (the test of the consistency of $a \cdot \alpha + \beta$) to pass unless the first set of shares decodes to a (this being a 2-universal hash function [CW79] (also see e.g. [GN93, Sec. 3.1]); in Section 7 we provide a proof of a more general statement as explained below). This way, the correctness threshold is raised to n , but the robustness threshold still remains zero.

One can envision using a better code than $\mathbf{G}^*$ in the above authentication-augmented sharing of SPDZ. Let’s say we use a linear code $\mathbf{G}^{(d)}$ that is known to have minimum distance d and has an efficient error-correcting procedure ECC. Thus, $\mathbf{G}^{(d)}$ has rank $k \leq n-d+1$ (see e.g. [vL91, Lemma 5.2.2]). Reed-Solomon codes [vL91, Section 6.8], being maximum distance separable codes, actually achieve this bound, so let $k = n-d+1$. Then, this code achieves privacy against $k-1 = n-d$ parties, and the robustness threshold increases to $\lfloor \frac{d-1}{2} \rfloor$. However, the correctness threshold decreases to $n-d$ (since at that point, the secrecy of α and β is lost). Taking $d = 2n/3$, we get privacy, correctness and robustness against

⁴This should be contrasted with list-decoding procedures [Sud97].

⁵An example of such an (efficient) procedure is the Berlekamp-Welch decoder for Reed-Solomon codes (see e.g. [GS92b]).

⁶In SPDZ, the α and β shares are also further authenticated using pre-processed private keys.

$< n/3$ parties (which is already achieved by the BGW protocol, which uses the same Reed-Solomon code without any need for augmented authentication). If we set $d = n/2$, we note that this methodology (augmenting the shares with additional shares for authentication) can potentially get the privacy and correctness thresholds to $n/2$. However, the robustness threshold remains low at $\approx n/4$. This is better than the BGW methodology using a rank $n/2$ Reed-Solomon code, which as remarked earlier can only get a correctness threshold marginally (or not at all) above the robustness threshold of $n/4$ (the privacy threshold remains $n/2$). Both these options are sub-optimal: (a) for the BGW case, we would like the correctness threshold to be higher than the privacy threshold, but it is too low at $n/4$, and (b) for SPDZ, the robustness threshold is too low (i.e. at $n/4$) while the correctness threshold is at the desirable level of $n/2$. One would like the robustness and correctness thresholds to be equal, as in some secure computations, it is rather important to have a computation complete correctly rather than being aborted – for example, an election or an auction that had a large popular build-up. In this work, we show how to achieve *fast* online-stage MPC with the privacy threshold at $n/3$ (as in BGW), but with the robustness and correctness thresholds vastly improved to $n/2$.

The main idea of our protocol is to use two Reed-Solomon codes of different ranks for sharing the underlying secrets. The strategy is to use a Reed-Solomon code of rank $n/3$ to share the secret a and a Reed-Solomon code of rank $n/2$ to share the authentication components $a \cdot \alpha + \beta$, α , and β . There are two main problems with enacting this strategy. First, proving that a single affine element β prevents a rushing attack is challenging. However, we will show that a vector of n independent affine elements can be used to provably prevent rushing attacks with no extra communication as compared to a single affine element. Second, to achieve a better decoding threshold for the rank $n/2$ code (than the usual $n/4$), we need to expand the code to be a block code over $\mathbb{F}^{c \cdot n}$, where c is some constant, i.e. each component of the codeword is itself in $\mathbb{F}^{c \cdot n}$ (sometimes referred to as folded codes [Kra03]). The first promising idea is to use the bivariate code of BGW [BGW88] (also see [AL17]). In this code, to encode a , instead of choosing a univariate polynomial $f(X)$ of degree $n/2 - 1$ with constant term being a , a bivariate polynomial $\mathcal{U}(X, Y)$ of degree $n/2 - 1$ (in each variable) is chosen with constant term a . Each component of the codeword, say the i -th component, is in $\mathbb{F}^n$ (i.e. $c = 1$), represented by two degree $n/2 - 1$ univariate polynomials $u_i(X)$ and $v_i(Y)$ which are respectively $\mathcal{U}(X, \zeta_i)$ and $\mathcal{U}(\zeta_i, Y)$. Thus, the diagram of this probabilistic encoding is $\mathbb{F} \xrightarrow{\$} \mathbb{F}^{n/2 \times n/2} \rightarrow (\mathbb{F}^{n/2 + n/2})^n$. A remarkable fact is that the error-correcting distance of this code is $n/2 - 1$, far beyond the usual $n/4 - 1$ for rank $n/2$ Reed-Solomon codes. More precisely, say for even n , for a received and potentially erroneous codeword $\{u_i^*(X), v_i^*(Y)\}_{i \in [n]}$, if for a subset of indices $I \subseteq [n]$ of size at least $n/2 + 1$, it is the case that for all $i, j \in I$, $u_i^*(\zeta_j) = v_j^*(\zeta_i)$ and vice versa, then there is a unique codeword at Hamming distance $\leq n/2 - 1$ (see e.g. [AL17, Lemma 5.3]). Thus, if there are $\leq n/2 - 1$ errors, which would imply the above sized clique I , then there is a unique codeword at Hamming distance $\leq n/2 - 1$. Unfortunately, since finding cliques is NP-hard, this

does not yield an efficient decoding algorithm, and it remains an open problem to devise an efficient decoding algorithm for this code.

Fortunately, if the code employs secret evaluation points, say $w_1, \dots, w_n$, with one evaluation point being secretly given to each server (in pre-processing), then a slightly expanded bivariate code (with $c = 3/2$) has easy decoding. This idea first appeared in [DMO⁺11], again in the context of pre-processing MPC (also see [BPRW16, FY19, EF23] where a slightly different but equivalent formulation of such codes is called robust codes). This code is best understood as a concatenated code, and the resulting randomized sharing as a concatenated sharing, i.e. *a sharing of a sharing*. Thus, to encode $a \in \mathbb{F}$, it is first encoded using Shamir sharing, i.e. with diagram $\mathbb{F} \xrightarrow{\mathbb{S}} \mathbb{F}^d \rightarrow \mathbb{F}^n$, where the (second) map $\mathbf{G}$ is a degree $(d-1)$ Reed-Solomon encoding using public evaluation points $\zeta_1, \dots, \zeta_n$. Next, each component of this sharing is again Shamir shared with diagram $\mathbb{F} \xrightarrow{\mathbb{S}} \mathbb{F}^{n/2} \rightarrow \mathbb{F}^{n/2+n}$, where the second code (map) is a systematic code (i.e. includes the message) with parity check elements that are a degree $(n/2-1)$ Reed-Solomon encoding using secret evaluation points $w_1, \dots, w_n$. If we view the secret evaluation points also as part of the encoding, then this would make it a non-linear code. For this reason, the secret points are considered separately distributed and used across many encodings (if the encodings are needed to satisfy linearity). Of course, such robust secret-sharing schemes have rather expensive preprocessing. We present a better novel robust code next (see Fig 2), which has the advantage that the preprocessing cost is reduced by a factor of n .

A Concatenated Sharing Scheme $[[\cdot]]^{\mathbb{F}, \mathbb{E}, n, d}$

Assume $\mathbb{E}$ is an extension field of $\mathbb{F}$. On input $a \in \mathbb{F}$, do the following:

1. Generate fresh random degree $d-1$ Shamir-shares of a over $\mathbb{F}$ using public evaluation points $\zeta_1, \dots, \zeta_n \in \mathbb{F}$ (these points can be in sequence starting with 1). Call these shares $[a]$, and consider it as an n -vector over $\mathbb{F}$.
2. Choose n evaluation points $w_i \in \mathbb{E}$ ($i \in [n]$) uniformly and independently at random, conditioned on all n points being non-zero and distinct.
3. For each $i, j \in [n]$, choose a random coefficient $b_{ij} \in \mathbb{E}$. Let $v_{ij}(X) = [a]_i + b_{ij}X$ be an affine polynomial over $\mathbb{E}$.
4. The i -th share of $[[a]]^{\mathbb{F}, \mathbb{E}, n, d}$ consists of the following: (i) share $[a]_i$, (ii) coefficients $\{b_{ij}\}_{j \neq i \in [n]}$, (iii) evaluations $\{v_{ji}(w_i)\}_{j \neq i \in [n]}$, and (iv) the evaluation point w_i .

Fig. 2

In this work, we combine this concatenated sharing scheme with SPDZ-style authenticators to obtain the best of both worlds in terms of robustness and correctness thresholds, a fast online MPC protocol, and *single round* verifiable input sharing by clients, both in the pre-processing model.

1.8 Related Work

We have already discussed several relevant related works above, but here we give a more thorough review of the related literature in honest majority MPC with robustness (not necessarily with pre-processing). In the $t < n/3$ setting, [BGW88] and concurrently [CCD88] showed that robust protocols with perfect security are possible. [HMP00] introduced the player elimination framework, and [BTH08, GLS19] achieved linear communication complexity in this setting, but required $\Omega(D + n)$ rounds when considering both offline and online phases. [AAY21, AAPP23] showed how to obtain $O(D)$ rounds while maintaining good communication complexity. In the $t < n/2$ setting, the works [BTH06, BFO12, GSZ20, CP17, EF23, ABDE24] have focused on improving the communication and round complexity. In particular, [ABDE24] achieves honest-majority robust MPC with $O(D)$ rounds while improving on prior works in terms of communication complexity when considering both the offline and online phases.

The works of [PS17, PS18] introduce the notion of responsiveness for protocols in the context of blockchains, and [LM18] discusses responsive protocols in the context of Byzantine agreement.

2 Preliminaries

Our MPC protocols operate over authenticated point-to-point channels. In our UC models, we assume that all participants know each others' identities and have authenticated point-to-point channels with each other. We omit session IDs throughout for ease of exposition. We assume a partially synchronous model, where messages sent by honest parties to other honest parties are guaranteed to arrive within Δ time steps, where Δ is a predetermined upper bound on the network latency. We prove our results in the UC model with statistical security against a malicious adversary with static corruptions.

2.1 Responsive Protocols

We say that a protocol (or a round of a protocol) is *responsive* if a party that receives messages in a round of a protocol from all parties (from which it expects a message) in time $\delta < \Delta$ can proceed to the next round. This is in contrast to non-responsive protocols, where a party must wait Δ time before proceeding to the next round, even if it has already received all the expected messages for a round. To illustrate via two examples, the online phase of BGW-style MPC is responsive whereas the attestation phase of SPDZ is non-responsive because it relies on a “commit and open” style paradigm for aggregation that necessitates that all commitments be sent to all parties before any openings.

2.2 Secret Sharing Preliminaries

Let $\mathbb{F}$ be a finite field over which the arithmetic circuit is defined. Let $\zeta_1, \zeta_2, \dots, \zeta_n$ be distinct non-zero elements of $\mathbb{F}$. We will assume $|\mathbb{F}| \geq n+1$ so that these distinct non-zero elements can be chosen.

A (d, n) -Shamir secret share of an element $a \in \mathbb{F}$ is the list of n elements $([a]_i = f(\zeta_i))_{i \in [n]}$ where $f(X)$ is a degree $d - 1$ polynomial defined over $\mathbb{F}$ with the constant term of $f(X)$ set to a . The full collection of n shares is denoted by $^d[a]$, or just $[a]$ when d is clear from context. The elements $\zeta_1, \zeta_2, \dots, \zeta_n$ are referred to as the **evaluation points** of the Shamir secret-sharing scheme (as well as the related Reed-Solomon code).

Using the above evaluation points, define the following Vandermonde matrix:

$$\mathbf{V}^{n \times d} = \begin{pmatrix} 1 & \zeta_1 & \dots & \zeta_1^{d-1} \\ 1 & \zeta_2 & \dots & \zeta_2^{d-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_n & \dots & \zeta_n^{d-1} \end{pmatrix}$$

Then, $^d[a] = \mathbf{V}^{n \times d} \cdot \mathbf{f}$ where $\mathbf{f}$ is the vector of coefficients of $f(X)$ with constant term a , i.e. $\mathbf{f}_0 = a$. It is well-known that any d rows of the above matrix form an invertible matrix. Hence, any d shares out of the n shares can be used to (efficiently) determine a . Note that Shamir secret-sharing is linear in that $[a] + [b] = [a + b]$, and $\lambda \cdot [a] = [\lambda \cdot a]$ for any $\lambda \in \mathbb{F}$.

When we say that $^d[a]$ is a **fresh random** Shamir secret-sharing of a , we mean that the vector $\mathbf{f}$ was chosen uniformly at random from $\mathbb{F}^d$ conditioned on $\mathbf{f}_0 = a$. In this work, we will mostly use two types of Shamir secret-shares, one with $d = n/3$ and another with $d = n/2$. If not otherwise mentioned, we will use the notation $[a]$ for $^{n/3}[a]$ and $\langle a \rangle$ for $^{n/2}[a]$.

Sometimes, the Shamir secret-shares are augmented with additional information. One such augmentation was defined in Figure 2. In such a case, we will use the notation $\llbracket a \rrbracket$ to refer to the augmented shares of a . If the augmentation is of $^{n/2}[a]$, i.e. $\langle a \rangle$, then we will use the notation $\langle\langle a \rangle\rangle$ to refer to the augmented shares. These augmented shares make use of an extension field $\mathbb{E}$ of $\mathbb{F}$. If the base field $\mathbb{F} = \mathbb{E}$ (that is, the secret $a \in \mathbb{E}$ and the Shamir secret sharing is done over $\mathbb{E}$), we will use the notation $\llbracket a \rrbracket^{\mathbb{E}}$ (resp., $\langle\langle a \rangle\rangle^{\mathbb{E}}$). The augmented secret-sharing scheme defined in Figure 2 uses secret evaluation points $w_1, \dots, w_n$ that are sampled during share generation. Sometimes, we will require these secret evaluation points to be the same across multiple sharings. In this case, we will explicitly mention this and use the notation $\tilde{\theta}$ to refer to the vector of secret evaluation points $\theta_1, \dots, \theta_n$ that are common across sharings.

2.3 Opening Sharings for $t < n/3$

We overview the procedure from [DN07] for n parties $P_1, \dots, P_n$ to robustly open consistent degree $t < n/3$ Shamir secret shares in the presence of a malicious adversary controlling at most t parties. Each party P_i possesses share x_i of some secret x . We first describe an opening procedure that relies on a chosen party P^* .

Open($P^*, t, [x]$)

1. Each party P_i sends its share x_i of $[x]$ to P^* .
2. P^* runs Berlekamp-Welch on the received shares to recover x and sends x to all parties.

Since $t < n/3$, it follows that there are at least $2t + 1$ correct shares and at most t incorrect shares given to P^* . Thus, Berlekamp-Welch [GS92a] will correctly decode x . Of course, the above opening procedure has the issue that it requires P^* to be honest. To handle this issue, [DN07] give an opening procedure, that simultaneously opens $\ell = n - (2t + 1)$ sharings by first expanding them into n sharings using a linear error correcting code. Each party is tasked with opening one sharing and then the dishonest openings can be corrected. The opening procedure is given below.

OpenRobust($t, [x^{(1)}], \dots, [x^{(\ell)}]$)

1. Each party P_i applies an $n \times \ell$ Vandermonde matrix to its shares $(x_i^{(1)}, \dots, x_i^{(\ell)})$ to obtain shares $y_i^{(1)}, \dots, y_i^{(\ell)}$.
2. For each $i \in [n]$, run (in parallel) **Open**($P_i, t, [y^{(i)}]$).
3. All parties run Berlekamp-Welch on received values $y^{(1)}, \dots, y^{(\ell)}$ to obtain $x^{(1)}, \dots, x^{(\ell)}$.

Since affine combinations of degree t Shamir sharings produce degree t Shamir sharings, the $y_i^{(j)}$'s are correct sharings of $y^{(j)}$. Since there are at most $t < n/3$ dishonest parties, at most t of the received $y^{(j)}$'s will be incorrect, and so Berlekamp-Welch will correctly decode $x^{(1)}, \dots, x^{(\ell)}$.

3 A Verifiable Secret Sharing Scheme

We will use the verifiable secret sharing scheme $\llbracket \cdot \rrbracket^{\mathbb{F}, \mathbb{E}, n, d}$ defined in Fig. 2 throughout this paper. We now describe the corresponding ideal functionality $\mathcal{F}_{\text{VSS-PRE}}$ that gives out correlated randomness to the n servers.

Ideal Functionality $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$

Parameters: A field $\mathbb{F}$, a finite extension field $\mathbb{E}$ of $\mathbb{F}$, and a $d - 1$ degree Shamir secret-sharing scheme $[\cdot]$ over $\mathbb{F}$.

Participants: An Originator O and n servers $S_1, \dots, S_n$.

Generate/Expand: On receiving message (Generate) or (Expand, $^d[a]$) from O , do the following:

1. If the originator did not provide an $^d[a]$ (i.e. called with (Generate)), then set $a \xleftarrow{\$} \mathbb{F}$ and run the code of the concatenated sharing scheme $\llbracket \cdot \rrbracket^{\mathbb{F}, \mathbb{E}, n, d}$ with input $a \in \mathbb{F}$ and output $\llbracket a \rrbracket_i$ to each server S_i .

2. If the originator provided $d[a]$ (i.e. called with (Expand)), then run the code of $\llbracket \cdot \rrbracket^{\mathbb{F}, \mathbb{E}, n, d}$ with the initial shares $[a]$ set to those provided by the originator and output $\llbracket a \rrbracket_i$ to each server S_i

We now describe the protocol to open, verify, and reconstruct the shared a .

Protocol $\Pi_{\text{VSS}}.\text{OpenSound}$

Input: Each server S_i has its share of $\llbracket a \rrbracket$. Thus, server S_i has $[a]_i$, $\{b_{ij}\}_{j \neq i}$, $\{v_{ji}(w_i)\}_{j \neq i}$, and w_i . Recall that $v_{ji}(X) = [a]_j + b_{ji}X$.

Open-Start: Each server S_i sends to each server S_j its polynomial $v_{ij}(X)$.

Verification and Reconstruction: Each server S_i receives from S_j a polynomial, say $\tilde{v}_{ji}(X)$. If $\tilde{v}_{ji}(w_i) \neq v_{ji}(w_i)$ (recall S_i has $v_{ji}(w_i)$ and w_i), then reject server S_j . If $\tilde{v}_{ji}(w_i) = v_{ji}(w_i)$, then accept server S_j and let the constant term $\tilde{v}_{ji}(0)$ be $[a]_j^{(i)}$. Since $[\cdot]$ is a Shamir secret-sharing scheme using degree $d-1$ polynomials, if at least $d-1$ servers pass the test, then S_i uses any valid $d-1$ shares and its own share $[a]_i$ to get the constant term of the degree $d-1$ polynomial, say $a^{(i)}$. Output $a^{(i)}$. If less than $d-1$ servers pass the test, then output $\perp$.

It is clear that up to $d-1$ corrupt and colluding servers do not get any information about a before the opening (either from their shares of $[a]$ or from the evaluations of honest servers' $v_{ji}(X)$). This can be proven using a simulation argument, though we will anyway prove this as part of our bigger protocols (see Lemma 5.1). Further, we have the following correctness lemma.

Lemma 3.1. *In the protocol $\Pi_{\text{VSS}}.\text{OpenSound}$ on input $\llbracket a \rrbracket$, if at least d servers are honest, then each honest server outputs a , except with probability upper bounded by $(n-d)/(|\mathbb{E}| - n)$.*

Proof. If at least d servers are honest, then every honest server S_i receives valid shares from at least $d-1$ servers, which would all pass the verification test and hence be accepted by S_i , in which case $\tilde{v}_{ji}(X) = v_{ji}(X)$ for every server S_j that is honest. Hence for honest servers, $[a]_j^{(i)} = [a]_j$. However, we need to rule out that a dishonest server, say S_k , does not manage to give S_i a polynomial $\tilde{v}_{ki}(X)$, where $\tilde{v}_{ki}(X) \neq v_{ki}(X)$, but $\tilde{v}_{ki}(w_i) = v_{ki}(w_i)$, as this would cause S_i to erroneously accept S_k . But, since w_i is random in $\mathbb{E}$, and all quantities given to every server other than S_i are independent of w_i other than the fact that the w_i 's are distinct and nonzero, the probability that the non-zero $\tilde{v}_{ki}(X) - v_{ki}(X)$ (a degree one polynomial) has the root w_i is at most $1/(|\mathbb{E}| - n)$. By a union bound, the probability of any of the $n-d$ dishonest servers succeeding is at most $(n-d)/(|\mathbb{E}| - n)$.

Remark. If the same secrets w_i are used over many sessions, say ℓ many, then the probability that an honest server is fooled in any session by any malicious server is at most $\ell(n-d)/(|\mathbb{E}| - n)$.

The following additive-homomorphism property of $\llbracket \cdot \rrbracket$ is worth noting. Let $\llbracket a \rrbracket_{\text{lin}, i}$ denote all of $\llbracket a \rrbracket_i$ except the secret evaluation point w_i . That is, $\llbracket a \rrbracket_i = (\llbracket a \rrbracket_{\text{lin}, i}, w_i)$.

Lemma 3.2. *For two secrets a, b shared with $\llbracket \cdot \rrbracket$ using the same verification secrets w_i for each server S_i and a value $\lambda \in \mathbb{F}$, it holds that $\llbracket a \rrbracket_{\text{lin}} + \lambda \cdot \llbracket b \rrbracket_{\text{lin}} = \llbracket a + \lambda \cdot b \rrbracket_{\text{lin}}$.*

Proof. This follows immediately from the linearity of $[\cdot]$ and the fact that $(v_{ij}^{(1)}(X) + \lambda \cdot v_{ij}^{(2)}(X))(w_j) = v_{ij}^{(1)}(w_j) + \lambda \cdot v_{ij}^{(2)}(w_j)$.

When we operate with different sharings of $\llbracket \cdot \rrbracket$ using the same verification secrets w_i , we will omit lin for brevity of notation and simply write $\llbracket a \rrbracket + \lambda \cdot \llbracket b \rrbracket = \llbracket a + \lambda \cdot b \rrbracket$ for the above.

We have a further aggregation property using the Schwartz-Zippel lemma.

Lemma 3.3. *For any $m = \text{poly}(n)$, and arbitrary values $\{r^{(\ell)}\}_{\ell \in [m]}$ and $\{s^{(\ell)}\}_{\ell \in [m]}$ in field $\mathbb{E}$, given their $\llbracket \cdot \rrbracket$ shares, let x be a random and independent value in $\mathbb{E}$. Then, if the shares of party i of $\sum_{\ell \in [m]} x^\ell \cdot \llbracket r^{(\ell)} \rrbracket$ are accepted by j as shares of $\sum_{\ell \in [m]} x^\ell \cdot \llbracket s^{(\ell)} \rrbracket$ with non-negligible probability, then for all $\ell \in [m]$, shares of party i of $\llbracket r^{(\ell)} \rrbracket$ will be accepted by party j as shares of $\llbracket s^{(\ell)} \rrbracket$ with probability one.*

Proof. For party j to accept these shares, it must be the case that

$$\sum_{\ell \in [m]} x^\ell \cdot v_{ij}^{(r, \ell)}(w_j) = \sum_{\ell \in [m]} x^\ell \cdot v_{ij}^{(s, \ell)}(w_j)$$

with non-negligible probability. Thus, by the Schwartz-Zippel lemma, for all $\ell \in [m]$, $v_{ij}^{(r, \ell)}(w_j) = v_{ij}^{(s, \ell)}(w_j)$, which completes the proof.

4 Ideal Functionality for Extra-Robust Secure-Function-Evaluation (X-SFE)

In this section, we define an ideal functionality for secure function evaluation (SFE) in the multi-client distributed-server setting. Secure function evaluation has been studied extensively and various UC definitions exist covering even Byzantine and adaptive settings and feasibility of such definitions have been considered in both synchronous and asynchronous communication models (see [Can00] and references therein for these definitions). In this work, we consider variations of these definitions which can lead to much more efficient realizations, especially in an offline-online setting. The variations add some new features and exclude some features normally considered. The exclusions are mostly for ease of exposition, and the results presented here can mostly be extended to cover these exclusions as well.

Ideal Functionality $\mathcal{F}_{\text{X-SFE}[\mathbb{F}, f, n, m]}$

Participants: A maximum of m users P_i ($i \in [m]$), a global coordinator GC, and adversary $\mathcal{S}$ interact with functionality $\mathcal{F}_{\text{X-SFE}}$.

Local Variables: `inputCommitted` (Boolean, initialized: false), `numServersCorrupted` (Integer, initialized: zero, max: n), array M of size m initialized to $\perp$.

Input Phase: On receiving (inp, i, x) from P_i : If `numServersCorrupted` $\in [n/3, n]$ then send (inp, i, x) to $\mathcal{S}$ else send (inp, i) to $\mathcal{S}$. Next, if `inputCommitted` then ignore else set $M[i] = x$.

startCompute: On receiving message `startCompute` from GC:

- If `numServersCorrupted` $\in [0, n/2 - 1]$ then set `inputCommitted` to true, and compute $y = f(M)$ and output y to GC and $\mathcal{S}$ and exit.
- If `numServersCorrupted` $> n/2$ then send `startCompute` to $\mathcal{S}$. When $\mathcal{S}$ responds back with (output, y') output y' to GC and exit.

Corruption:

- On receiving a message $(\text{corruptOneMoreServer}, i)$ from $\mathcal{S}$ increment `numServersCorrupted`. If `numServersCorrupted` $\in [n/3, n]$ then give (current) M to $\mathcal{S}$. If `numServersCorrupted` $> n/2$, then $\mathcal{S}$ can alter M to M' (before `startCompute`), as well direct to send $\perp$ to GC.
- If $\mathcal{S}$ corrupts a user P_i , then send $M[i]$ to $\mathcal{S}$. Also, if $\mathcal{S}$ provides a x' (including $\perp$) and `inputCommitted` is false, then set $M[i] = x'$.

Fig. 3: An ideal functionality for multi-client distributed-server extra-robust secure function evaluation (X-SFE) for function f over $\mathbb{F}$ with at most m inputs. The functionality captures the security and privacy requirements of computing f on inputs of m clients using n servers. The global coordinator GC can, for instance, be a global clock (for issuing `startCompute`) and a public bulletin-board for publishing the result.

Some exclusions worth mentioning are (a) *probabilistic* function evaluation, (b) different users getting different functions' evaluations (of the same inputs) [guaranteed output delivery of this setting can typically be only realized in the synchronous communication model (see [GL90, Bea91b])], and (c) peer-to-peer SFE.

The focus of this work is on multi-client distributed-server SFE. In other words, many clients (for simplicity, we will assume that their identities are known in advance) have inputs, and a function f with one common output needs to be computed on these inputs. Instead of the clients handing their inputs to a trusted party who could then evaluate the function f and return the result to all valid users (ideal world), we would like the trusted party be emulated by a set of known n servers. So, this is indeed secure-function evaluation except that the n servers do not evaluate on their private inputs but on inputs of some known clients. It is important to consider how the input phase will be terminated so that the computation can begin. We assume there is a global clock or a global

coordinator that determines the input phase’s stopping point. The real-world implementation of these mechanisms is beyond the scope of this work.

The novelty of our definition lies in the fact that we require robustness/GOD and correctness to be guaranteed for a higher threshold of corrupted parties as compared to the threshold required for guaranteeing privacy of the clients’ inputs. The ideal functionality $\mathcal{F}_{\text{X-SFE}}$ is described in Fig 3.

Let M be the vector containing inputs from all participating clients (that were provided during the input phase before the stopping point). This may include both honest and malicious clients, but the malicious clients can only either set their input to be in the domain or explicitly set to $\perp$ (one can view this guarantee as a property of the ideal functionality– see “robustness against bad client” below). The security and privacy guarantees of functionality $\mathcal{F}_{\text{X-SFE}}$ can be summarized as follows:

Privacy: $< n/3$ malicious (corrupt) servers cannot learn anything other than the output $y = f(M)$.

Robustness: Even if $< n/2$ servers are maliciously faulty/corrupt, the other honest servers will compute and output $f(M)$.

Correctness: $< n/2$ malicious servers cannot cause honest servers to output to $y' \neq y$, where $y = f(M)$. Note this is implied by robustness for the same corruption threshold.

Censorship Resistance: $< n/2$ malicious servers cannot disqualify an honest client. Recall M is the vector of all inputs.

Robustness Against a Bad Client:

- A cheating client P_i can only input its value to be in the domain or explicitly set to $\perp$, i.e. in that case $M[i] = \perp$.
- If some client P_i is corrupt and colluding with $< n/2$ cheating servers, then the honest servers will compute and output $y = f(M)$.

5 Input Sharing

In this section, we describe how clients can share their inputs amongst the servers in our MPC protocol. At a high level, this is done by first having the servers run a preprocessing step where each server S_i receives a share $[\rho]_i$ for a random $\rho \in \mathbb{F}$ according to the concatenated sharing scheme of Figure 2. Once this preprocessing is finished, each server S_i sends $[\rho]_i$ to the client P , from which P can emulate $\Pi_{\text{VSS.OpenSound}}$ for each server to obtain $\rho^{(i)}$. If a majority of the $\rho^{(i)}$ ’s are equal to some ρ , then P will Byzantine broadcast $x - \rho$ to the servers, where x is its input. From this, each server can obtain $[x]_i$ with respect to the underlying Shamir secret-sharing scheme.

5.1 Ideal Functionality for Input Sharing

We first define an ideal functionality for a client P to secret share its input amongst n servers. We will assume that the client P and the n servers have

identities known to each other, as well as secure point-to-point communication channels. Typically, this is known in the literature as the verifiable secret sharing (VSS) ideal functionality ([FM97, BOGW88]) for which a universally-composable definition is formulated in [Can01]. However, we want a formulation that is efficiently realizable even when the robustness and correctness thresholds are $n/2$ corruptions whereas the sharing is only $n/3$ private; recall the shares are used in the MPC protocol that is $n/3$ private, but $n/2$ robust and correct.

Ideal Functionality $\mathcal{F}_{\text{inputShare}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]}$

Parameters: A field $\mathbb{F}$, the number of servers n , thresholds t_p , t_r , and t_c , and a sharing scheme $[\cdot]$ for n parties.

Participants: A client (dealer) P , n servers S_k ($k \in [n]$), and an adversary $\mathcal{S}$ interact with functionality $\mathcal{F}_{\text{inputShare}}$.

Local Variables: `numServersCorrupted` (Integer, Initialized: 0, Max: n), y (Initialized: $\perp$), $[y]$ (Initialized: $\perp^n$), `PCorrupt` (Boolean, Initialized: `false`).

Input: On receiving `(inp, P, x)` for $x \in \mathbb{F}$ from P : If `numServersCorrupted` $\geq t_p$ or `PCorrupt`, send `(inp, P, x)` to $\mathcal{S}$. Else, send `(inp, P)` to $\mathcal{S}$. Set $y = x$. Generate a fresh random $[y]$ and give $[y]_i$ to $\mathcal{S}$ for $i \in [\text{numServersCorrupted}]$.

Share: On receiving message `(Share, y')` from $\mathcal{S}$ for $y' \in \mathbb{F} \cup \{\perp\}$,

- If `PCorrupt` and $y' = \perp$, output $\perp$ to all honest servers.
- Else if `numServersCorrupted` $< t_r$: Output $[y]_i$ to each honest server S_i and `ok` to P .
- Else if $t_c > \text{numServersCorrupted} \geq t_r$: If $y' = y$, then output $[y]_i$ to each honest server S_i and `ok` to P . If $y' \neq y$, then output the message “`tooManyServersCorrupted`” to all honest servers and P .
- Else if `numServersCorrupted` $\geq t_c$: Output to each honest server and P as dictated by $\mathcal{S}$.

Corruption: On receiving message `corruptOneMoreServer` from $\mathcal{S}$, increment `numServersCorrupted` and give $[y]_{\text{numServersCorrupted}}$ to $\mathcal{S}$. If `numServersCorrupted` $\geq t_p$, give y to $\mathcal{S}$. On receiving message `(corruptUser, P)` from $\mathcal{S}$, set `PCorrupt` = `true`, and give y to $\mathcal{S}$. If P is already corrupted, $\mathcal{S}$ can also send a message `(reset, P, z)` in which case $\mathcal{F}_{\text{inputShare}}$ updates $[y]_i \leftarrow [y]_i + z - y$, sets $y = z$, and gives $[y]_i$ to $\mathcal{S}$ for $i \in [\text{numServersCorrupted}]$.

Fig. 4: An ideal functionality for resilient and verifiable input-sharing of a client’s input amongst n servers using general sharing $[\cdot]$ and privacy (t_p), robustness (t_r), and correctness (t_c) thresholds. The session IDs have been suppressed for readability.

This definition is motivated by likely scenarios where a set of servers are more incentivized to cause a wrong output (and hence more likely to be corruptible for this end) than to just learn the value being shared. Similarly, there are situations where a client is more concerned about its shared value y being misconstrued as $y' \neq y$ than y being leaked. Of course, there are situations where this is not the

case, but our justification is that in the preprocessing model, this higher level of robustness and correctness can be realized with small extra effort.

In fact, we can define three different thresholds for privacy (t_p), robustness (t_r) and correctness (t_c). To reiterate, the security and privacy guarantees can be informally summarized as follows:

Privacy: For $t < t_p$, t malicious (corrupt) servers obtain no information about y , the value shared by honest P .

Robustness: For $t < t_r$, even if t servers are maliciously faulty/corrupt, the other honest servers can reconstruct y , the value shared by honest P .

Client Verifiability: For $t < t_c$, if P is corrupt and t servers are corrupt (and/or colluding with P), then either P distributes $[z]$ for some z or all honest servers declare P corrupt, i.e. *disqualify* P .

Correctness: For $t < t_c$, if P is honest, then t malicious servers cannot cause honest servers to open (reconstruct) to $y' \neq y$, where y is the value shared by honest P , *nor* can these malicious servers cause honest servers to disqualify P .

While the above properties are informally described, these notions are formalized in the ideal functionality $\mathcal{F}_{\text{inputShare}}$ described in Fig 4. As opposed to having an “opening” phase as in the UC-definition of VSS in [Can01], this functionality is parameterized by a secret-sharing scheme and actually gives out shares to the specified servers.

5.2 UC Realization of $\mathcal{F}_{\text{inputShare}}$ in the Preprocessing Model

We now show that the ideal functionality $\mathcal{F}_{\text{inputShare}}$ can be realized for any $[\cdot]$ that is a Shamir secret-sharing scheme using degree $d-1$ polynomials for $d = t_p \leq t_r = t_c \leq n/2$ in the preprocessing model. Importantly, the preprocessing material is only given to the n servers and *not* the client(s). In other words, the client(s) is not involved at all in the preprocessing, and the client(s) does not come into the picture until the actual time of input submission. Also, this protocol is a *single round protocol*, where the client gets information from each of the servers and then the client replies back to the servers. We remark that this protocol is similar to the input sharing scheme of SPDZ [DPSZ12, Fig 1], except that here the client(s) is not part of the set of parties which have preprocessed data. Yet, if for each input providing client, there was a separate authenticator generated using preprocessing, servers can just send their respective authenticating key to this particular client.

In more detail, this scheme utilizes the VSS scheme described in Section 3. Let $\mathcal{F}_{\text{BB}}[t]$ be a broadcast ideal functionality from a client P to n servers which can tolerate up to $t-1$ Byzantine malicious servers⁷. We now describe the protocol $\Pi_{\text{inputShare}}$ realizing $\mathcal{F}_{\text{inputShare}}$ in the $\mathcal{F}_{\text{VSS-PRE}}$ and $\mathcal{F}_{\text{BB}}$ -hybrid model.

⁷Note that such a broadcast functionality can be realized with the client sending its message once to the other parties (servers), and from there on only servers are involved in a Byzantine consensus protocol.

Protocol $\Pi_{\text{inputShare}}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]$ using $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$ and $\mathcal{F}_{\text{BB}}[t_c]$

Participants: A client P and n servers S_k ($k \in [n]$).

Input: P receives its input $x \in \mathbb{F}$ from the environment.

Server Preprocessing: An originator invokes $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$. This results in the n servers getting populated by correlated randomness $[\rho]$ for a random $\rho \in \mathbb{F}$. More precisely, server S_i has $[\rho]_i$, $\{b_{ij}\}_{j \neq i}$, $\{v_{ji}(w_i)\}_{j \neq i}$, and w_i . Recall that $v_{ij}(X) = [\rho]_i + b_{ij}X$.

Server S_i : Server S_i sends $[\rho]_i$ to P .

P Verifies and Submits: P receives from every S_i a secret w_i^* , $n-1$ degree 1 polynomials $v_{ij}^*(X)$, and $n-1$ values $v_{j,i}^*$ (for $j \neq i$; purportedly $v_j(w_i)$). For each i , P now emulates S_i as in $\Pi_{\text{VSS.OpenSound}}$'s verification and reconstruction step. Each such emulation leads to either server S_i rejecting (output $\perp$) or outputting a $\rho^{(i)}$. If a majority of (emulated) S_i 's $\rho^{(i)}$ are equal to some ρ , then P uses $\mathcal{F}_{\text{BB}}[t_c]$ to (Byzantine) broadcast $x - \rho$ to the n servers. If not (i.e. no ρ reached majority), then P aborts.

Server S_i : On receiving some u in the broadcast from P , output $[\rho]_i + u$ as its share.

Theorem 5.1. *For $d = t_p \leq t_r = t_c \leq n/2$ where $[\cdot]$ is a degree $d-1$ Shamir secret-sharing scheme, the above protocol $\Pi_{\text{inputShare}}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]$ UC-realizes $\mathcal{F}_{\text{inputShare}}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]$ in the $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$, $\mathcal{F}_{\text{BB}}[t_c]$ -hybrid model with statistical distance $\leq (n-d)/(|\mathbb{E}| - n)$ as long as the total number of servers corrupted is less than t_c .*

A description of the simulator that interacts with the ideal functionality $\mathcal{F}_{\text{inputShare}}$, as well as a proof of simulation for the above theorem can be found in Supplementary Material A.

6 A Novel $n/3$ -Private SFE Protocol with Honest-Majority Robustness and Correctness

In this section, we describe a protocol that UC-realizes the extra-robust secure-function evaluation functionality $\mathcal{F}_{\text{X-SFE}}$ of Section 4. Since this protocol will use global-authenticator a la SPDZ, where a global secret α is used to pre-authenticate various Beaver tuples, the input sharing must also use the same global authenticator α . In other words, the input sharing scheme, such as the one that realizes $\mathcal{F}_{\text{inputShare}}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]$ and gives shares to servers using the $[\cdot]$ parameterized sharing-scheme, must employ a sharing scheme that uses α -authenticated Beaver tuples. We now describe the preprocessing ideal functionality, which we call $\mathcal{F}_{\text{X-Beaver}}$. As is usual in offline-online MPC protocols, this preprocessing functionality just generates various Beaver multiplication tuples, as well as authenticated versions of these using the global MAC key α . As previously mentioned, it also generates sharings to be used by the input phase

by calling $\mathcal{F}_{\text{VSS-PRE}}$. This functionality can be implemented by any of the known MPC protocols that are secure against up to $n/2$ malicious corruptions, such as [GSZ20, CP17, ABDE24]. We give such an instantiation in Supplementary Material C.

Ideal Functionality $\mathcal{F}_{\text{X-Beaver}}[\mathbb{F}, \mathbb{E}, n, \ell, c]$

Qualification: Let $\mathbb{E}$ be a finite extension of $\mathbb{F}$ with $|\mathbb{E}| > (\ell + c) \cdot n^2 \cdot 2^\lambda$, where λ is the statistical security parameter.

Participants: An originator $\mathcal{O}$ and n servers $S_1, \dots, S_n$.

Generate: On receiving message (Generate) from $\mathcal{O}$, $\mathcal{F}$ does the following:

1. Pick n evaluation points $\theta_i \in \mathbb{E}$ ($i \in [n]$) uniformly and independently at random, conditioned on all n points being non-zero and distinct. Output θ_i to S_i . Call this array of points $\vec{\theta}$.
2. Sample random non-zero $\alpha, x \xleftarrow{\$} \mathbb{E}$. Generate fresh random $\langle\langle \alpha \rangle\rangle^{\mathbb{E}}$ and $\langle\langle x \rangle\rangle^{\mathbb{E}}$ using evaluation points $\vec{\theta}$ and distribute them (share-wise) to the n servers.
3. For $k = 1$ to ℓ do:
 - i. Sample $\rho_k \xleftarrow{\$} \mathbb{F}$.
 - ii. Generate fresh random $[\rho_k]$ (note: as per our default convention, these are degree $n/3 - 1$ Shamir shares).
 - iii. Call $\mathcal{F}_{\text{VSS-PRE}}$ with (Expand, $[\rho_k]$) to get $\llbracket \rho_k \rrbracket$ along with its evaluation points w_i (note: $\llbracket \rho_k \rrbracket_i$ contains w_i).
 - iv. Generate fresh random $\langle\langle \rho_k \cdot \alpha \rangle\rangle^{\mathbb{E}}$ using the evaluation points $\vec{\theta}$.
 - v. Distribute $(\llbracket \rho_k \rrbracket, \langle\langle \rho_k \cdot \alpha \rangle\rangle^{\mathbb{E}})$ (share-wise) to the n servers. This tuple will be referred to as an **input-converting authenticated tuple**.
4. For $k = 1$ to c do:
 - i. Pick $l_k, r_k \xleftarrow{\$} \mathbb{F}$.
 - ii. Generate fresh random $[l_k], [r_k]$, and $[l_k r_k]$ and distribute them (share-wise) to the n servers (note: as per our default convention, these are degree $n/3 - 1$ Shamir shares).
 - iii. Generate fresh random $\langle\langle l_k \cdot \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle r_k \cdot \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle l_k r_k \cdot \alpha \rangle\rangle^{\mathbb{E}}$, all of these using the same evaluation points $\vec{\theta}$. Distribute these (share-wise) to the n servers.
 - iv. The complete set $[l_k], [r_k], [l_k r_k], \langle\langle l_k \cdot \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle r_k \cdot \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle l_k r_k \cdot \alpha \rangle\rangle^{\mathbb{E}}$ will be referred to as an **extended and authenticated Beaver tuple**. The first part, i.e. $[l_k], [r_k], [l_k r_k]$, will be referred to as a **basic Beaver tuple**.
5. Pick a random n -vector $\vec{\beta} = \langle \beta_1, \dots, \beta_n \rangle$ in $\mathbb{E}$. Modify the encoding scheme $\llbracket \cdot \rrbracket^{\mathbb{E}, \mathbb{E}, n, n/2}$ described in Figure 2, where instead of setting $v_{ij}(0)$ to be $[a]_i$, now $v_{ij}(0)$ is set to be β_i . The resulting codeword, generated using the evaluation points $\vec{\theta}$, will be written as $\langle\langle \vec{\beta} \rangle\rangle^{\mathbb{E}}$. Distribute the shares of this codeword to the n servers.

Having described the preprocessing ideal functionality $\mathcal{F}_{\text{X-Beaver}}$, we now describe the online protocol in the $\mathcal{F}_{\text{X-Beaver}}$ -hybrid model. It has a similar structure to that of the BGW online protocol using Beaver tuples, but it extends these tuples with authenticated tuples, which are used to later check in an attestation phase, a la SPDZ, whether more than $n/3$ (but $< n/2$) malicious servers caused an incorrect circuit evaluation. The scheme achieves an $n/3$ robustness threshold as it employs an underlying degree $n/3 - 1$ Shamir secret-sharing. However, this robustness threshold can be improved to $n/2$ by noting that if the attestation phase fails, then it is guaranteed that more than $n/3$ servers are malicious, and so privacy is lost anyways. Therefore, the servers can simply open all inputs and compute the function in the clear in this instance, which extends the robustness threshold to $n/2$. The authenticated tags and global MAC key α are encoded using the concatenated code of Fig. 2 which assures decoding with even up to $< n/2$ corruptions (with overwhelming probability). The scheme is described next, and it is self-explanatory.

Protocol $\Pi_{\text{ONLINE-X-SFE}[\mathbb{F}, f, n, m]}$

Setup. Let the function f be represented by an arithmetic circuit C over $\mathbb{F}$, with a maximum of m inputs. A global coordinator GC invokes $\mathcal{F}_{\text{X-Beaver}}[\mathbb{F}, \mathbb{E}, n, m, |C| + 1]$ to populate the n servers with

- Secrets $\theta_i \in \mathbb{E}$
- Input-converting authenticated tuples (one for each input wire)
- Extended and authenticated Beaver tuples (one for each multiplication gate in C and the output wire)
- $\langle\langle \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle x \rangle\rangle^{\mathbb{E}}, \langle\langle \vec{\beta} \rangle\rangle^{\mathbb{E}}$

A maximum of m clients $P_1, P_2, \dots, P_m$ share their inputs amongst the n servers using $\Pi_{\text{inputShare}}[\mathbb{F}, n, \langle n/3, n/2, n/2 \rangle, [\cdot]]$ (which itself uses input-converting authenticated tuples generated by $\mathcal{F}_{\text{X-Beaver}}[\mathbb{F}, \mathbb{E}, n, m, |C| + 1]$ above replacing the stand-alone invocation of $\mathcal{F}_{\text{VSS-PRE}}$). Let $\ell \leq m$ be the number of clients that succeed in sharing their inputs. Observe that if client P_i has input a_i , then during the execution of $\Pi_{\text{inputShare}}[\mathbb{F}, n, \langle n/3, n/2, n/2 \rangle, [\cdot]]$, the servers use the $\llbracket \rho_i \rrbracket$ portion of the input-converting authenticated tuple $(\llbracket \rho_i \rrbracket, \langle\langle \rho_i \cdot \alpha \rangle\rangle^{\mathbb{E}})$ and obtain $[a_i]$. Moreover, all servers also obtain the value $\hat{a}_i = a_i - \rho_i$ that is Byzantine broadcast by P_i . Letting $\rho'_i = -\rho_i$ and noting that $\langle\langle \rho'_i \cdot \alpha \rangle\rangle^{\mathbb{E}} = -\langle\langle \rho_i \cdot \alpha \rangle\rangle^{\mathbb{E}}$, it follows that all servers have $\hat{a}_i = a_i + \rho'_i$ and their shares of $[a_i]$ and $\langle\langle \rho'_i \cdot \alpha \rangle\rangle^{\mathbb{E}}$.

Online Stage. The servers proceed in $|C|$ stages for the $|C|$ multiplication gates. [More efficiently, the servers can handle all gates at the same depth simultaneously as that saves on communication when using **OpenRobust**.]

The computation will be described inductively. As the base case, which are the input wires, for each input a_i , the servers have the value $\hat{a}_i = a_i + \rho_i$ and shares of $[a_i], \langle \rho_i \cdot \alpha \rangle^{\mathbb{E}}$.

Inductively, assume that the servers have the value $\hat{a}_i = a_i + \rho_i$ and shares of $[a_i], \langle \rho_i \cdot \alpha \rangle^{\mathbb{E}}$ for an intermediate set of wires $\mathbb{I}$ that feed into an addition gate g . Addition is performed locally as follows:

Procedure Add($\{[a_i], \hat{a}_i, \langle \rho_i \cdot \alpha \rangle^{\mathbb{E}}\}_{i \in \mathbb{I}} :$

1. Compute $[a_g] = \sum_{i \in \mathbb{I}} [a_i]$, $\hat{a}_g = \sum_{i \in \mathbb{I}} \hat{a}_i$, $\langle \rho_g \cdot \alpha \rangle^{\mathbb{E}} = \sum_{i \in \mathbb{I}} \langle \rho_i \cdot \alpha \rangle^{\mathbb{E}}$
2. Return $[a_g], \hat{a}_g, \langle \rho_g \cdot \alpha \rangle^{\mathbb{E}}$.

Assume that a specific multiplication gate $g \in [\ell + 1.. \ell + |C|]$ has one left input a_l , and one right input a_r , and outputs $a_g = a_l \cdot a_r$. Assume that the servers have the values $\hat{a}_l = a_l + \rho_l, \hat{a}_r = a_r + \rho_r$, and the shares $[a_l], \langle \rho_l \cdot \alpha \rangle^{\mathbb{E}}, [a_r], \langle \rho_r \cdot \alpha \rangle^{\mathbb{E}}$. They access an unused extended and authenticated Beaver tuple and compute $[a_g]$ as follows:

Procedure Mult($[a_l], \hat{a}_l, \langle \rho_l \cdot \alpha \rangle^{\mathbb{E}}, [a_r], \hat{a}_r, \langle \rho_r \cdot \alpha \rangle^{\mathbb{E}} :$

1. Obtain a fresh extended and authenticated Beaver tuple, say

$$[l], [r], [lr], \langle l \cdot \alpha \rangle^{\mathbb{E}}, \langle r \cdot \alpha \rangle^{\mathbb{E}}, \langle lr \cdot \alpha \rangle^{\mathbb{E}}.$$

2. Compute $[a_l + l], [a_r + r]$.
3. Open $a_l + l, a_r + r$ using **OpenRobust**.
4. Compute $[a_g] = (a_l + l)(a_r + r) - (a_l + l)[r] - (a_r + r)[l] + [lr]$.
5. Compute $\langle \rho_g \cdot \alpha \rangle^{\mathbb{E}} = (a_l + l)\langle r \cdot \alpha \rangle^{\mathbb{E}} + (a_r + r)\langle l \cdot \alpha \rangle^{\mathbb{E}} - \langle lr \cdot \alpha \rangle^{\mathbb{E}}$.
6. Compute $\langle P_g \rangle^{\mathbb{E}} = \langle \rho_l \cdot \alpha \rangle^{\mathbb{E}} - \langle l \cdot \alpha \rangle^{\mathbb{E}}$, $\langle Q_g \rangle^{\mathbb{E}} = \langle \rho_r \cdot \alpha \rangle^{\mathbb{E}} - \langle r \cdot \alpha \rangle^{\mathbb{E}}$.
7. Compute $\hat{a}_g = (a_l + l)(a_r + r)$.
8. Compute $R_g = \hat{a}_l - (a_l + l)$, and $S_g = \hat{a}_r - (a_r + r)$.
9. Return $[a_g], \hat{a}_g, \langle \rho_g \cdot \alpha \rangle^{\mathbb{E}}, \langle P_g \rangle^{\mathbb{E}}, R_g, \langle Q_g \rangle^{\mathbb{E}}, S_g$

(*Remark:* While $[a_g], \hat{a}_g, \langle \rho_g \cdot \alpha \rangle^{\mathbb{E}}$ are used iteratively, the rest are saved for the attestation phase.)

Output Wire. For the output wire w , open $[a_w + l]$, using an authenticated Beaver tuple but ignore $[r]$ etc. Similarly, compute R_w and ignore computing S_w , as well as compute $\langle P_w \rangle^{\mathbb{E}}$ and ignore $\langle Q_w \rangle^{\mathbb{E}}$. In the attestation below, treat Q_w and S_w for this output wire as zero.

Attestation Phase.

- i. The n servers open $\langle x \rangle^{\mathbb{E}}$ with $\Pi_{\text{VSS}}.\text{OpenSound}$ (of Section 3).

ii. Now the servers locally compute:

$$\begin{aligned}\langle\langle A \rangle\rangle^{\mathbb{E}} &= \sum_{g=\ell+1}^{\ell+|C|+1} x^{2(g-\ell)} \langle\langle P_g \rangle\rangle^{\mathbb{E}} + x^{2(g-\ell)+1} \langle\langle Q_g \rangle\rangle^{\mathbb{E}}, \\ B &= \sum_{g=\ell+1}^{\ell+|C|+1} x^{2(g-\ell)} R_g + x^{2(g-\ell)+1} S_g.\end{aligned}$$

- iii. Recall, each server S_i 's share of $\langle\langle A \rangle\rangle^{\mathbb{E}}$ consists of (i) $\langle A \rangle_i$, (ii) coefficients $\{b_{ij}\}_{j \neq i}$, and (iii) $\{v_{ji}(\theta_i)\}_{j \neq i}$. Each server S_i computes $v'_{ij}(X) := \beta_i + v_{ij}(X)$. Recall $v_{ij}(X) = \langle A \rangle_i + b_{ij}X$ and $\beta_i \in \mathbb{E}$ is part of $\langle\langle \tilde{\beta} \rangle\rangle^{\mathbb{E}}$ generated in step 5 of $\mathcal{F}_{X\text{-Beaver}}$. Server S_i sends $v'_{ij}(X)$ to server S_j . This step is same as the Open-Start part of $\Pi_{\text{VSS.OpenSound}}$, except that $v_{ij}(X)$ has been replaced by $v'_{ij}(X)$.
- iv. The n servers open $\langle\langle \tilde{\beta} \rangle\rangle^{\mathbb{E}}$ with $\Pi_{\text{VSS.OpenSound}}$. Each server S_i either gets β_j or $\perp$ (for all $j \in [n]$). For each server S_i , let J_i be the set of servers $j \in [n]$ that had the β_j verified by $\Pi_{\text{VSS.OpenSound}}$. For all $j \in J_i$, each server S_i updates its received polynomial $v'_{ji}(X)$ (in the previous step) by subtracting β_j from it, and saves it as $v_{ji}(X)$. For $j \notin J_i$, it sets $v_{ji}(X) = \perp$. Each server now runs the verification and reconstruction part of $\Pi_{\text{VSS.OpenSound}}$ (for $\langle\langle A \rangle\rangle^{\mathbb{E}}$) to get A .
- v. The n servers engage with the other servers to open α from $\langle\langle \alpha \rangle\rangle^{\mathbb{E}}$ using $\Pi_{\text{VSS.OpenSound}}$. If $\alpha = 0$, the server aborts.
- vi. Each server checks whether $A \stackrel{?}{=} B \cdot \alpha$. If the check fails, set $\hat{a}_{\ell+|C|}$ to $\perp$.
- vii. Servers engage with other servers to open $l \cdot \alpha$, where l is as in the output wire description, from $\langle\langle l \cdot \alpha \rangle\rangle^{\mathbb{E}}$ using $\Pi_{\text{VSS.OpenSound}}$. Next, each server divides by α to get l and then computes $a_{\ell+|C|} = \hat{a}_{\ell+|C|} - l$ (if $\hat{a}_{\ell+|C|}$ was $\perp$ then $a_{\ell+|C|} = \perp$). If $a_{\ell+|C|}$ is not $\perp$, the server outputs this value to GC.
- viii. Servers engage in an honest-majority Byzantine consensus on $a_{\ell+|C|}$, and output the consensus a (if note $\perp$) to GC (see comment below).
- ix. IF consensus is $\perp$, the parties re-run the protocol, this time using robust Beaver tuples $\langle\langle l \cdot \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle r \cdot \alpha \rangle\rangle^{\mathbb{E}}, \langle\langle lr \cdot \alpha \rangle\rangle^{\mathbb{E}}$. Note, that since the parties have α , they can adjust the third component of the triple to be $\langle\langle lr \cdot \alpha^2 \rangle\rangle^{\mathbb{E}}$.

GC: Once GC (asynchronously) receives a value that has frequency at least $n/2$, it outputs this value to environment $\mathcal{E}$. If it receives enough responses so that it is assured that no one value will get frequency $n/2$, it outputs $\perp$ to the environment. Note that if no party misbehaved or erred, then in step (vii) itself more than $n/2$ parties would have output the same value to GC, and the environment would have got the output from GC even before the step (viii)'s Byzantine consensus was complete.

7 UC Security Proof

Theorem 7.1. *The protocol $\Pi_{\text{ONLINE-X-SFE}[\mathbb{F}, f, n, m]}$ UC-realizes ideal functionality $\mathcal{F}_{\text{X-SFE}[\mathbb{F}, f, n, m]}$ in the $\mathcal{F}_{\text{X-Beaver}}$, $\mathcal{F}_{\text{inputShare}}$, $\mathcal{F}_{\text{BC-hybrid}}$ model, where $\mathcal{F}_{\text{BC}}$ denotes the ideal functionality for honest-majority Byzantine consensus.*

The simulator for the proof of this theorem as well as the full proof of the above theorem can be found in Supplementary Material B. However, the most technical innovation in this proof can be abstracted in the following lemma.

Lemma 7.2. [Correctness] *When $n_{\text{bad}} \in [n/3, n/2 - 1]$, if an honest server's input to the honest-majority Byzantine consensus in step viii of the attestation phase is not $\perp$, then with overwhelming probability it is $f(M)$.*

Proof. If an honest server's input to the honest-majority Byzantine consensus in the attestation phase is not $\perp$, then the check $A \stackrel{?}{=} B \cdot \alpha$ passed for this server. We will show that if the check $A \stackrel{?}{=} B \cdot \alpha$ passes, then with overwhelming probability, the $\hat{a}$ value of the output obtained by this server is correct (meaning it is the $\hat{a}$ value that would be obtained during an honest execution of the protocol). By the correctness of $\Pi_{\text{VSS.OpenSound}}$ (Lemma 3.1), it follows that α and $l \cdot \alpha$ are correctly opened with overwhelming probability, and this server's input to the honest-majority Byzantine consensus is $\hat{a} - l = f(M)$.

We now show that if the check $A \stackrel{?}{=} B \cdot \alpha$ passes, then the $\hat{a}$ value obtained by this server S_i is correct with overwhelming probability. We first observe that since $n_{\text{bad}} < n/2$, x , α , and β are opened correctly with overwhelming probability. Let A_{hon} and B_{hon} denote the values of A and B that would be obtained during the attestation phase of the honest protocol execution. Let A_i and B_i denote the values actually obtained by S_i . Let J denote the set of servers $j \in [n]$ that S_i uses shares from to reconstruct A_i . S_i will use its own share, so note that $i \in J$. Let $A^{(j)}$ denote the j th share of A obtained by S_j during the protocol execution. Let $A_{\text{hon}}^{(j)}$ denote the values of these shares during the honest execution. It follows for reconstruction coefficients λ_j that

$$A_i = \sum_{j \in J} \lambda_j A^{(j)}$$

and

$$A_{\text{hon}} = \sum_{j \in J} \lambda_j A_{\text{hon}}^{(j)}.$$

Let $J_{\text{hon}} \subseteq J$ be the set of honest servers in J and let $J_{\text{corr}} \subseteq J$ be the set of corrupted servers in J . For honest servers $j \in J_{\text{hon}}$, we can similarly define $P_g^{(j)}$ and $Q_g^{(j)}$ as the shares of P_g and Q_g , respectively, used by S_j to compute $A^{(j)}$. For all servers $j \in J$, we define $P_{g, \text{hon}}^{(j)}$ and $Q_{g, \text{hon}}^{(j)}$ analogously as the shares obtained during the honest protocol execution. For $j \in J_{\text{hon}}$, define $P_g^{(j)} = P_{g, \text{hon}}^{(j)} + \delta_g^{(j)}$ and

$Q_g^{(j)} = Q_{g,\text{hon}}^{(j)} + \epsilon_g^{(j)}$. For $j \in J_{\text{corr}}$, define $\delta_g^{(j)} = \epsilon_g^{(j)} = 0$. It follows that

$$A_i = \sum_{j \in J_{\text{corr}}} \lambda_j \Delta^{(j)} + \sum_{j \in J} \lambda_j \sum_{g=\ell+1}^{\ell+|C|} x^{2(g-\ell)} (P_{g,\text{hon}}^{(j)} + \delta_g^{(j)}) + x^{2(g-\ell)+1} (Q_{g,\text{hon}}^{(j)} + \epsilon_g^{(j)}),$$

where $\Delta^{(j)}$ denotes the arbitrary difference of the j th share determined by the adversary (note that this models the fact that the adversarial $\Delta^{(j)}$ may not be a low degree polynomial in x). Observe that $\Delta^{(j)}$ is independent of α . This follows since the view of the adversary when it sends $v'_{ji}(X)$ thus committing to its share $A^{(j)}$ is independent of α since the honest parties' shares are masked by β_i 's. Define $R_{g,\text{hon}}^{(i)}$ and $S_{g,\text{hon}}^{(i)}$ as the values of R_g and S_g that server S_i would obtain in the honest execution of the protocol, and $R_g^{(i)}$ and $S_g^{(i)}$ as the values actually obtained by server S_i . Define $R_g^{(i)} = R_{g,\text{hon}}^{(i)} + \tau_g^{(i)}$ and $S_g^{(i)} = S_{g,\text{hon}}^{(i)} + \psi_g^{(i)}$. Observe that the $\tau_g^{(i)}$'s and $\psi_g^{(i)}$'s are independent of α since they can be determined by the adversary whose view during the protocol execution is independent of α . Furthermore, note that the $\delta_g^{(j)}$'s and $\epsilon_g^{(j)}$'s can be expressed in the form $c \cdot \alpha$ for some c independent of α . This follows by noting that any error introduced in computing $P_g^{(j)}$ is determined by the computed value of $\langle \rho_l \cdot \alpha \rangle^{\mathbb{E}}$, which is determined by the openings $a_{l'} + l'$ and $a_{r'} + r'$ (l' and r' denote the left and right indices of the gate with output l) which are known to the adversary whose view is independent of α . Define $\delta_g'^{(j)}$ (resp. $\epsilon_g'^{(j)}$) as $\delta_g^{(j)}/\alpha$ (resp. $\epsilon_g^{(j)}/\alpha$).

Suppose $A_i = B_i \cdot \alpha$. Then,

$$\sum_{j \in J_{\text{corr}}} \lambda_j \Delta^{(j)} + \sum_{j \in J} \lambda_j \sum_{g=\ell+1}^{\ell+|C|} x^{2(g-\ell)} (P_{g,\text{hon}}^{(j)} + \delta_g^{(j)}) + x^{2(g-\ell)+1} (Q_{g,\text{hon}}^{(j)} + \epsilon_g^{(j)}) = B_i \cdot \alpha,$$

which implies

$$\sum_{j \in J_{\text{corr}}} \lambda_j \Delta^{(j)} = \alpha \cdot \left[\sum_{g=\ell+1}^{\ell+|C|} x^{2(g-\ell)} (\tau_g^{(i)} - \sum_{j \in J_{\text{hon}}} \lambda_j \delta_g'^{(j)}) + x^{2(g-\ell)+1} (\psi_g^{(i)} - \sum_{j \in J_{\text{hon}}} \lambda_j \epsilon_g'^{(j)}) \right].$$

The above expression is of the form $W = \alpha \cdot V$ with both W and V independent of α . Thus, with overwhelming probability, $W = V = 0$. By the Schwartz-Zippel lemma, it follows that with overwhelming probability,

$$\sum_{g=\ell+1}^{\ell+|C|} X^{2(g-\ell)} (\tau_g^{(i)} - \sum_{j \in J_{\text{hon}}} \lambda_j \delta_g'^{(j)}) + X^{2(g-\ell)+1} (\psi_g^{(i)} - \sum_{j \in J_{\text{hon}}} \lambda_j \epsilon_g'^{(j)}) = 0$$

as a polynomial and thus all coefficients are 0. This holds because x is independent of all the coefficients of this polynomial since it is not opened until they have all been determined. Thus,

$$\forall g \in [\ell + 1.. \ell + |C|], \tau_g^{(i)} - \sum_{j \in J_{\text{hon}}} \lambda_j \delta_g'^{(j)} = 0 \text{ and } \psi_g^{(i)} - \sum_{j \in J_{\text{hon}}} \lambda_j \epsilon_g'^{(j)} = 0. \quad (1)$$

We now show inductively that all the errors are 0 and thus $A_i = A_{\text{hon}}$ and $B_i = B_{\text{hon}}$. First, due to the correctness of the input phase (Lemma 5.1), it follows that honest servers have the correct shares $([a], \hat{a}, \langle \rho \cdot \alpha \rangle^{\mathbb{E}})$ for input wires. By correct shares, we mean that these shares match the values of those in the honest execution of the protocol. It is immediate that if all input shares to an addition gate are correct, then the shares of the output gate are also correct. We induct sequentially over the multiplication gates. For the base case, consider the first multiplication gate $g = \ell + 1$. Since the input wires are correct, it follows that the inputs to this multiplication gate g are correct. Thus, $P_g^{(j)}$ and $Q_g^{(j)}$ are correct and so $\delta_g^{(j)}$ and $\epsilon_g^{(j)}$ are 0 for all $j \in J_{\text{hon}}$. By (1), this implies $\tau_g^{(i)} = \psi_g^{(i)} = 0$, which implies that $a_l + l$ and $a_r + r$ were opened correctly and thus, $([a_g], \hat{a}_g, \langle \rho_g \cdot \alpha \rangle^{\mathbb{E}})$ are correctly computed by server S_i .

Now consider any multiplication gate $g \in [\ell + 1.. \ell + |C|]$. By the induction hypothesis, S_i computed the correct shares for the inputs to this multiplication gate. It immediately follows that $P_g^{(i)}$ and $Q_g^{(i)}$ are computed correctly by S_i and so $\delta_g^{(i)} = \epsilon_g^{(i)} = 0$. For any $j \in J_{\text{hon}}$, since S_i accepted $A^{(j)}$ as consistent with its share $A^{(i)}$, it follows from Lemma 3.3 that $P_g^{(j)}$ and $Q_g^{(j)}$ are consistent with $P_g^{(i)}$ and $Q_g^{(i)}$, respectively. Therefore, by Lemma 3.1, $\delta_g^{(j)} = \epsilon_g^{(j)} = 0$ for all $j \in J_{\text{hon}}$ with overwhelming probability. By (1), this implies $\tau_g^{(i)} = \psi_g^{(i)} = 0$, which implies that $a_l + l$ and $a_r + r$ were opened correctly and thus, $([a_g], \hat{a}_g, \langle \rho_g \cdot \alpha \rangle^{\mathbb{E}})$ are correctly computed by server S_i .

Thus, for all $g \in [\ell + 1.. \ell + |C|]$, $\tau_g^{(i)} = \psi_g^{(i)} = 0$, which implies that the entire computation is computed correctly by S_i , and the output $\hat{a}$ obtained by server S_i is correct with overwhelming probability.

References

- AAPP23. Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Detect, pack and batch: Perfectly-secure mpc with linear communication and constant expected time. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 251–281, Cham, 2023. Springer Nature Switzerland.
- AAY21. Ittai Abraham, Gilad Asharov, and Avishay Yanai. Efficient perfectly secure computation with optimal resilience. In Kobbi Nissim and Brent Waters, editors, *Theory of Cryptography*, pages 66–96, Cham, 2021. Springer International Publishing.
- ABDE24. Amit Agarwal, Alexander Bienstock, Ivan Damgård, and Daniel Escudero. Honest Majority GOD MPC with $O(\text{depth}(C))$ Rounds and Low Online Communication. Cryptology ePrint Archive, Paper 2024/1479, 2024.
- AL17. Gilad Asharov and Yehuda Lindell. A full proof of the bgw protocol for perfectly secure multiparty computation. *J. Cryptol.*, 30(1):58–151, January 2017.
- BCD⁺09. Peter Bogetoft, Dan Lund Christensen, Ivan Damgård, Martin Geisler, Thomas Jakobsen, Mikkel Krøigaard, Janus Dam Nielsen, Jesper Buus Nielsen, Kurt Nielsen, Jakob Pagter, Michael I. Schwartzbach, and Tomas

- Toft. Secure multiparty computation goes live. In Roger Dingledine and Philippe Golle, editors, *FC 2009*, volume 5628 of *LNCS*, pages 325–343, February 2009.
- BDOZ11. Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 169–188, May 2011.
- Bea91a. Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology—CRYPTO’91*, pages 420–432. Springer, 1991.
- Bea91b. Donald Beaver. Secure multiparty protocols and zero-knowledge proof systems tolerating a faulty minority. *J. Cryptol.*, 4(2):75–122, 1991.
- BFO12. Eli Ben-Sasson, Serge Fehr, and Rafail Ostrovsky. Near-linear unconditionally-secure multiparty computation with a dishonest minority. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 663–680, August 2012.
- BGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 1–10. ACM, 1988.
- BOGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pages 1–10, 1988.
- BPRW16. Allison Bishop, Valerio Pastro, Rajmohan Rajaraman, and Daniel Wichs. Essentially optimal robust secret sharing with maximal corruptions. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT*, volume 9665 of *Lecture Notes in Computer Science*, pages 58–86. Springer, 2016.
- BTH06. Zuzana Beerliová-Trubíniová and Martin Hirt. Efficient multi-party computation with dispute control. In Shai Halevi and Tal Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 305–328, March 2006.
- BTH08. Zuzana Beerliová-Trubíniová and Martin Hirt. Perfectly-secure MPC with linear communication complexity. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 213–230, March 2008.
- Can00. Ran Canetti. Security and composition of multiparty cryptographic protocols. *J. Cryptol.*, 13(1):143–202, 2000.
- Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. *ECCC*, TR01-016, 2001. <https://eccc.weizmann.ac.il/eccc-reports/2001/TR01-016>.
- CCD88. David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In *20th ACM STOC*, pages 11–19. ACM Press, May 1988.
- CGH⁺18. Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 34–64, August 2018.
- CP17. Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Trans. Inf. Theor.*, 63(1):428–468, January 2017.

- CR03. Ran Canetti and Tal Rabin. Universal composition with joint state. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO*, volume 2729 of *Lecture Notes in Computer Science*, pages 265–281. Springer, 2003.
- CW79. Larry Carter and Mark N. Wegman. Universal classes of hash functions. *J. Comput. Syst. Sci.*, 18(2):143–154, 1979.
- DMO⁺11. Rafael Dowsley, Jörn Müller-Quade, Akira Otsuka, Goichiro Hanaoka, Hideki Imai, and Anderson C. A. Nascimento. Universally composable and statistically secure verifiable secret sharing scheme based on pre-distributed data. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 94-A(2):725–734, 2011.
- DN07. Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007.
- DPSZ12. Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Annual Cryptology Conference*, pages 643–662. Springer, 2012.
- EF23. Daniel Escudero and Serge Fehr. On fully-secure honest majority MPC without n^2 round overhead. LNCS, pages 47–66, 2023.
- FM97. Peasech Feldman and Silvio Micali. An optimal probabilistic protocol for synchronous byzantine agreement. *SIAM J. Comput.*, 26(4):873–933, 1997.
- FY19. Serge Fehr and Chen Yuan. Towards optimal robust secret sharing with security against a rushing adversary. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019*, volume 11478 of *Lecture Notes in Computer Science*, pages 472–499. Springer, 2019.
- GIP⁺. Daniel Genkin, Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*.
- GIP15a. Daniel Genkin, Yuval Ishai, and Antigoni Polychroniadou. Efficient multiparty computation: From passive to active security via secure SIMD circuits. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of LNCS, pages 721–741, August 2015.
- GIP⁺15b. Daniel Genkin, Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. *IACR Cryptol. ePrint Arch.*, page 154, 2015.
- GL90. Shafi Goldwasser and Leonid A. Levin. Fair computation of general functions in presence of immoral majority. In Alfred Menezes and Scott A. Vanstone, editors, *Advances in Cryptology - CRYPTO '90, 10th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1990, Proceedings*, volume 537 of *Lecture Notes in Computer Science*, pages 77–93. Springer, 1990.
- GLS19. Vipul Goyal, Yanyi Liu, and Yifan Song. Communication-efficient unconditional MPC with guaranteed output delivery. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of LNCS, pages 85–114, August 2019.
- GMW87. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.

- GN93. Peter Gemmell and Moni Naor. Codes for interactive authentication. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO '93*, volume 773 of *Lecture Notes in Computer Science*, pages 355–367. Springer, 1993.
- GRR98. Rosario Gennaro, Michael O. Rabin, and Tal Rabin. Simplified vss and fast-track multiparty computations with applications to threshold cryptography. In *Proceedings of the Seventeenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '98, 1998.
- GS92a. P. Gemmell and M. Sudan. Highly resilient correctors for polynomials. *Information processing letters*, 43(4):169–174, 1992.
- GS92b. Peter Gemmell and Madhu Sudan. Highly resilient correctors for polynomials. *Inf. Process. Lett.*, 43(4):169–174, 1992.
- GSZ20. Vipul Goyal, Yifan Song, and Chenzhi Zhu. Guaranteed output delivery comes free in honest majority MPC. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 618–646, August 2020.
- HMP00. Martin Hirt, Ueli M. Maurer, and Bartosz Przydatek. Efficient secure multi-party computation. In Tatsuaki Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 143–161, December 2000.
- KK06. Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 445–462, August 2006.
- Kra03. Victor Yu Krachkovsky. Reed-solomon codes for correcting phased error bursts. *IEEE Transactions on Information Theory*, 49(11):2975–2984, 2003.
- LM18. Julian Loss and Tal Moran. Combining asynchronous and synchronous byzantine agreement: The best of both worlds. *Cryptology ePrint Archive*, 2018.
- LN17. Yehuda Lindell and Ariel Nof. A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 259–276. ACM Press, October / November 2017.
- NP06. Moni Naor and Benny Pinkas. Oblivious polynomial evaluation. *SIAM Journal on Computing*, 35.5:1254–1281, (2006).
- NYS. NYSE. NYSE Opening and Closing Auctions.
- PS17. Rafael Pass and Elaine Shi. Hybrid Consensus: Efficient Consensus in the Permissionless Model. In Andréa Richa, editor, *31st International Symposium on Distributed Computing (DISC 2017)*, volume 91 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 39:1–39:16, 2017.
- PS18. Rafael Pass and Elaine Shi. Thunderella: Blockchains with optimistic instant confirmation. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 3–33, April / May 2018.
- RB89. Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority (extended abstract). In David S. Johnson, editor, *Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA*, pages 73–85. ACM, 1989.
- Sud97. Madhu Sudan. Decoding of reed solomon codes beyond the error-correction bound. *J. Complex.*, 13(1):180–193, 1997.

A Proof of Theorem 5.1 (Input Sharing)

Theorem 5.1 (repeated) *For $d = t_p \leq t_r = t_c \leq n/2$ where $[\cdot]$ is a degree $d-1$ Shamir secret-sharing scheme, the above protocol $\Pi_{\text{inputShare}}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]$ UC-realizes $\mathcal{F}_{\text{inputShare}}[\mathbb{F}, n, \langle t_p, t_r, t_c \rangle, [\cdot]]$ in the $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$, $\mathcal{F}_{\text{BB}}[t_c]$ -hybrid model with statistical distance $\leq (n-d)/(|\mathbb{E}| - n)$ as long as the total number of servers corrupted is less than t_c .*

We first give a description of the simulator $\mathcal{S}$ that interacts with the ideal functionality $\mathcal{F}_{\text{inputShare}}$. We will use $\mathcal{F}$ to refer to $\mathcal{F}_{\text{inputShare}}$ and Π to refer to $\Pi_{\text{inputShare}}$ for simplicity of notation. WLOG, we will assume $\mathcal{A}$ corrupts the first $\text{numServersCorrupted}$ servers by index and use $t = \text{numServersCorrupted}$. Note that $t < t_c$.

Simulator $\mathcal{S}$

For every server S_i corrupted by $\mathcal{A}$, send `corruptOneMoreServer` to $\mathcal{F}$.

We handle simulation in three cases.

P Honest, $t \geq t_p$:

- $\mathcal{S}$ receives x from $\mathcal{F}$ along with $[x]_i$ for $i \in [t]$. Since $t \geq t_p$, it can compute $[x]_i$ for all $i \in [n]$.
- $\mathcal{S}$ samples a random $a \in \mathbb{F}$, sets $[y]_i = a + [x]_i$ for all $i \in [n]$ and runs the code of $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$ with $(\text{Expand}, [y])$ to obtain $\llbracket y \rrbracket$. For each $i \in [t]$, it sends $\llbracket y \rrbracket_i$ to S_i .
- When P broadcasts in Π , $\mathcal{S}$ sends (Share, x) to $\mathcal{F}$ and runs the code of $\mathcal{F}_{\text{BB}}[t_c]$ to broadcast $-a$ to the servers corrupted by $\mathcal{A}$.

P Honest, $t < t_p$:

- $\mathcal{S}$ receives $[x]_i$ for $i \in [t]$ from $\mathcal{F}$.
- $\mathcal{S}$ samples a random $a \in \mathbb{F}$ and sets $[y]_i = a + [x]_i$ for all $i \in [t]$. For all $i \in [n] \setminus [t]$, it sets $[y]_i$ uniformly at random. $\mathcal{S}$ runs the code of $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$ with $(\text{Expand}, [y])$ to obtain $\llbracket y \rrbracket$. For each $i \in [t]$, it sends $\llbracket y \rrbracket_i$ to S_i .
- When P broadcasts in Π , $\mathcal{S}$ sends $(\text{Share}, 0)$ to $\mathcal{F}$ and runs the code of $\mathcal{F}_{\text{BB}}[t_c]$ to broadcast $-a$ to the servers corrupted by $\mathcal{A}$.

P Corrupt:

- $\mathcal{S}$ sends $(\text{corruptUser}, P)$ to $\mathcal{F}$ and receives x along with $[x]_i$ for $i \in [t]$ from $\mathcal{F}$.
- For each $i \in [n] \setminus [t]$, $\mathcal{S}$ samples consistent shares $[x]_i$ with x and the already known $[x]_j$ for $j \in [t]$. If $t \geq t_p$, these additional shares for the honest servers are already completely determined. For $t < t_p$, the linear system of equations is underdetermined, and $\mathcal{S}$ satisfies it by choosing a solution uniformly at random and then computing the additional shares.

- $\mathcal{S}$ samples a random $a \in \mathbb{F}$ and sets $[y]_i = a + [x]_i$ for all $i \in [n]$.
- $\mathcal{S}$ runs the code of $\mathcal{F}_{\text{VSS-PRE}}[\mathbb{F}, \mathbb{E}, [\cdot]]$ with $(\text{Expand}, [y])$ to obtain $\llbracket y \rrbracket$. It emulates Π by sending $\llbracket y \rrbracket_i$ for each $i \in [t]$ to S_i and for each $i \in [n] \setminus [t]$ to P .
- If P runs $\mathcal{F}_{\text{BB}}[t_c]$ with value z , $\mathcal{S}$ checks if $z = -a$. If so, $\mathcal{S}$ sends (Share, x) to $\mathcal{F}$. Otherwise, $\mathcal{S}$ sends $(\text{reset}, P, z + a + x)$ and then $(\text{Share}, z + a + x)$ to $\mathcal{F}$.
- Else if P aborts, $\mathcal{S}$ sends $(\text{Share}, \perp)$ to $\mathcal{F}$.

WLOG, we will assume that the adversary $\mathcal{A}$ in the UC protocol $\Pi_{\text{inputShare}}$ is a dummy adversary and essentially passes back and forth commands and messages from the environment $\mathcal{E}$. Thus, from now on, we will use the environment $\mathcal{E}$ as the real adversary. We will show that the view of $\mathcal{E}$ interacting with $\mathcal{F}_{\text{inputShare}}$ and the simulator $\mathcal{S}$ described above is statistically indistinguishable from the view of $\mathcal{E}$ interacting with $\Pi_{\text{inputShare}}$. We first show the following lemma.

Lemma A.1. *When P is honest and $t < t_c \leq n/2$, in the real world execution of $\Pi_{\text{inputShare}}$, P will abort with probability $\leq (n-d)/(|\mathbb{E}| - n)$.*

Proof. This follows immediately from Lemma 3.1 which states that if at least $d = t_p$ servers are honest, then all honest servers output a except with probability $\leq (n-d)/(|\mathbb{E}| - n)$. Since $t_c \leq n/2$, this implies $n - t_c \geq n/2 \geq t_p$. This means that in P 's emulation, all honest servers output a except with probability $\leq (n-d)/(|\mathbb{E}| - n)$. Since $t < n/2$, a majority of servers are honest and output a , and so P does not abort except with probability $\leq (n-d)/(|\mathbb{E}| - n)$.

We now handle each corruption case separately.

Case P Honest, $t \geq t_p$: For uniform $a \in \mathbb{F}$, a and $x + a$ are identically distributed, so $\mathcal{E}$'s view up to pre-processing step is identical in the real and ideal worlds. In the real world protocol ρ is to be seen as a replacement for a used by the Simulator. Similarly, $x - a$ and $-a$ are identically distributed. In Π , P 's verification check fails with probability $\leq (n-d)/(|\mathbb{E}| - n)$, so we don't have perfect simulation. P 's broadcast and resulting shares of all parties are identically distributed by the same argument. So, $\mathcal{E}$'s view is identical in the real and ideal worlds as long as P 's verification check passes.

Case P Honest, $t < t_p$: For uniform $a \in \mathbb{F}$, a and $x + a$ are identically distributed. Moreover, since $t < t_p$, $[y]_i$ is information-theoretically hidden, so $\mathcal{E}$'s view up to pre-processing step is identical in the real and ideal worlds. In Π , P 's verification check fails with probability $\leq (n-d)/(|\mathbb{E}| - n)$, so we don't have perfect simulation. P 's broadcast and resulting shares of all parties are identically distributed by same the argument. So, $\mathcal{E}$'s view is identical in the real and ideal worlds as long as P 's verification check passes.

Case P Corrupt: For uniform $a \in \mathbb{F}$, a and $x + a$ are identically distributed, and the ideal functionality's generation of $[x]_i$'s are identically distributed to the simulator's when filling in remaining shares. So, all the pre-processing material given to $\mathcal{E}$ is identically distributed in the real and ideal worlds. P 's broadcast then determines the output of all servers, which are identically distributed in the real world to the ideal functionality regardless of choice. Thus, $\mathcal{E}$'s view is identically distributed in the real and ideal worlds.

Thus, in all cases, the statistical distance between $\mathcal{E}$'s view in the real and ideal worlds is $\leq (n - d)/(|\mathbb{E}| - n)$.

B Proof of Theorem 7.1 (ONLINE – X-SFE)

Theorem 7.1 (repeated) *The protocol $\Pi_{\text{ONLINE-X-SFE}[\mathbb{F}, f, n, m]}$ UC-realizes ideal functionality $\mathcal{F}_{\text{X-SFE}[\mathbb{F}, f, n, m]}$ in the $\mathcal{F}_{\text{X-Beaver}}$, $\mathcal{F}_{\text{inputShare}}$, $\mathcal{F}_{\text{BC-hybrid}}$ model, where $\mathcal{F}_{\text{BC}}$ denotes the ideal functionality for honest-majority Byzantine consensus.*

Proof. We will assume that the adversary $\mathcal{A}$ in the UC protocol Π is dummy, and essentially passes back and forth commands and messages from the environment $\mathcal{E}$. Thus, from now on we will use environment $\mathcal{E}$ as the real adversary, which outputs a single bit. The simulator $\mathcal{S}$ will also be the ideal world adversary for $\mathcal{F}_{\text{X-SFE}}$. It is a universal simulator that uses $\mathcal{A}$ as a black box.

We note that the description of $\Pi_{\text{ONLINE-X-SFE}}$ uses $\Pi_{\text{inputShare}}$ instead of $\mathcal{F}_{\text{inputShare}}$. This is because $\Pi_{\text{inputShare}}$ implements $\mathcal{F}_{\text{inputShare}}$ in the $\mathcal{F}_{\text{VSS-PRE-hybrid}}$ model (see Lemma 5.1). By the UC-composition theorem we could replace $\Pi_{\text{inputShare}}$ by $\mathcal{F}_{\text{inputShare}}$ in the description of $\Pi_{\text{ONLINE-X-SFE}}$, except that $\Pi_{\text{ONLINE-X-SFE}}$ itself uses pre-processing functionality $\mathcal{F}_{\text{X-Beaver}}$. Since $\mathcal{F}_{\text{X-Beaver}}$ and $\mathcal{F}_{\text{VSS-PRE}}$ use a common state, in particular a global (random) MAC key α and (random) evaluation points θ , one has to use the JUC-composition theorem, i.e the joint-sessions universal composability theorem proved in [CR03]. So, from now we will assume that $\Pi_{\text{ONLINE-X-SFE}}$ is a $\mathcal{F}_{\text{X-Beaver}}$, $\mathcal{F}_{\text{inputShare}}$, $\mathcal{F}_{\text{BC-hybrid}}$ protocol with joint state α, θ .

The simulator $\mathcal{S}$ is described next, which takes $\mathcal{A}$ as a blackbox, and interacting with ideal functionality $\mathcal{F}_{\text{X-SFE}}$ (as participant $\mathcal{S}$) simulates the view of $\mathcal{A}$ as in the real world. The main idea of the simulator is that it will emulate the ideal functionality $\mathcal{F}_{\text{X-Beaver}}$ and simulate $\mathcal{F}_{\text{inputShare}}$ as well as all of the honest servers in Π . Note that the parties in the real world protocol Π are a maximum of m clients P_j , a GC, and the n servers. However, since the m clients give their input shares using $\mathcal{F}_{\text{inputShare}}$, we will assume that the clients P_j ($j \in [m]$) are not participants of Π , but the n servers are pre-populated with shares of the $\ell \leq m$ inputs of the ℓ clients that succeeded in input phase. In other words, assuming there were ℓ valid inputs from clients ($\ell \leq m$), their inputs a_j are given to the n servers as $\hat{a}_j = a_j + \rho_j, [a_j], \langle \rho_j \cdot \alpha \rangle^{\mathbb{E}}$.

Simulator $\mathcal{S}$

The simulator $\mathcal{S}$ emulates the code of ideal functionality $\mathcal{F}_{\text{X-Beaver}}$, while also simulating the ℓ clients' shares $\hat{a}_j = a_j + \rho_j, [a_j], \langle \rho_j \cdot \alpha \rangle^{\mathbb{E}}$ (for $j \in 1..\ell$). We describe the simulation in the static corruption model, where the number and identity of corrupted servers is known at the outset. Suppose the set of servers corrupted is $\mathcal{C}$ and let $n_{\text{bad}} = |\mathcal{C}|$. $\mathcal{S}$ calls $\mathcal{F}_{\text{X-SFE}}$ n_{bad} times with `corruptOneMoreServer`, so that $\mathcal{F}_{\text{X-SFE}}$ has `numServersCorrupted` = n_{bad} .

Case $n_{\text{bad}} < n/3$. The simulator does not get the actual inputs M , but just the number of participants ℓ that have valid inputs (see the input phase of $\mathcal{F}_{\text{X-SFE}}$). So, it just runs the code of $\mathcal{F}_{\text{X-Beaver}}$ and also generates shares for the ℓ clients with all ℓ inputs set to zero. It gives the resulting data of the first n_{bad} servers to $\mathcal{A}$.

Next, it calls $\mathcal{F}$.`startCompute` and gets the output $y = f(M)$. Thereafter, the simulator executes the same code as Π for the honest servers treating $\mathcal{A}$ as a black-box for messages to and from corrupted servers except for one change: at step vii of the Attestation Phase, the simulator gives shares of the honest servers for the opening of $\langle l \cdot \alpha \rangle^{\mathbb{E}}$ (the l used in the final gate) so that these are consistent with the opening of $(l - f(M) + f(\vec{0})) \cdot \alpha$, as described in Lemma B.1.

Case $n_{\text{bad}} \geq n/3$. In this case, the ideal functionality $\mathcal{F}_{\text{X-SFE}}$ gives $\mathcal{S}$ the actual inputs M . The simulator then uses these actual inputs M instead of zero in the previous case of $n_{\text{bad}} < n/3$. Thus, the simulator executes the same code as Π for the honest servers treating $\mathcal{A}$ as a black-box for messages to and from corrupted servers. At the end of the protocol, all honest servers output some value y_j in the simulation. $\mathcal{S}$ also gets from $\mathcal{A}$ the values it is outputting for each corrupt server. If there is a value y' that has frequency $\geq n/2$ and $n_{\text{bad}} \geq n/2$, the simulator calls $\mathcal{F}_{\text{X-SFE}}$ with message `(output,  $y'$ )`. Else if $n_{\text{bad}} \geq n/2$, the simulator calls $\mathcal{F}_{\text{X-SFE}}$ with message `(output,  $\perp$ )`.

This finishes the description of the simulator.

Lemma B.1. *The simulator $\mathcal{S}$ which chooses the global authentication key α , the public evaluation points $\tilde{\zeta}$ (for the $\langle \cdot \rangle$ secret-sharing) and $\tilde{\theta}$ (for the $\langle \cdot \rangle$ secret-sharing), as well as random l for the final gate, and has access to only $f(M)$ where M is the clients' input, can efficiently compute an opening of the honest parties in attestation step vii so that it is consistent with an opening of $\langle (l - f(M) + f(\vec{0})) \cdot \alpha \rangle^{\mathbb{E}}$ instead of an opening of $\langle l \cdot \alpha \rangle^{\mathbb{E}}$ in the view of the adversary that has static-corrupted $< n/3$ servers.*

Proof. WLOG, assume that servers $1..n_{\text{bad}}$ are corrupt. Also, let $\vec{r}$ be the n_{bad} -shares of $\langle l \cdot \alpha \rangle$ given to $\mathcal{A}$, which are the constant terms of the polynomials $v_{ij}(X)$ given to $\mathcal{A}$ ($i \in [1..n_{\text{bad}}], j \neq i$). Let $\vec{V}$ be the top $n/2$ rows of Vandermonde matrix $\mathbf{V}^{n \times n/2}$ (using evaluation points $\zeta_1, \dots, \zeta_n$). Let $\vec{v}^\top$ be the top row of inverse of $\vec{V}$ (invertible, because it is Vandermonde). Then, letting $\vec{s}$ stand for the $n/2 - n_{\text{bad}}$

shares of the as many honest servers, we need

$$\vec{v}^\top \cdot \begin{pmatrix} \vec{r} \\ \vec{s} \end{pmatrix} = (l - f(M) + f(\vec{0})) \cdot \alpha$$

Since $n_{\text{bad}} < n/3 < n/2 - 1$, $\vec{s}$ has rank at least one. The simulator can use any one element from this (affine) subspace for $\vec{s}$, which together with $\vec{r}$, then determine all other shares of $\langle l \cdot \alpha \rangle$.

$\mathcal{S}$ also needs to give $v_{ij}(X)$ for all $i \in [n_{\text{bad}} + 1..n], j \neq i$, along with $v_{ji}(\theta_i)$ for all $i \in [n_{\text{bad}} + 1..n], j \neq i$. Recall, $\mathcal{A}$ already has $v_{ij}(X)$ and $v_{ji}(\theta_i)$ for $i \in [1..n_{\text{bad}}], j \neq i$. Let sh_i denote the i th (simulated) share of $\langle l \cdot \alpha \rangle$ previously determined by $\mathcal{S}$ and v_{ji}^* denote the evaluations $v_{ji}(\theta_i)$. For $i \in [n_{\text{bad}} + 1..n], j \in [1..n_{\text{bad}}]$, $\mathcal{S}$ determines b_{ij} such that $\text{sh}_i + b_{ij}\theta_j = v_{ij}^*$ and sets $v_{ij}(X) = \text{sh}_i + b_{ij}X$. It similarly sets $v_{ji}^* = v_{ji}(\theta_i)$. For $i \in [n_{\text{bad}} + 1..n], j \in [n_{\text{bad}} + 1..n]$ with $j \neq i$, $\mathcal{S}$ samples b_{ij} uniformly at random and sets $v_{ij}(X) = \text{sh}_i + b_{ij}X$ and $v_{ji}^* = v_{ji}(\theta_i)$.

We now describe a series of experiments between a probabilistic polynomial time challenger Chal and the environment $\mathcal{E}$, starting with Expt_0 which we describe next. We will show that the view of $\mathcal{E}$ in Expt_0 is same as its view in the ideal-world setting with $\mathcal{E}$ interacting with $\mathcal{F}_{\text{X-SFE}}$ and the simulator $\mathcal{S}$ described above. We end with an experiment which is identical to the real world execution of the protocol in Π . We will show that the environment $\mathcal{E}$ has negligible advantage in distinguishing between these series of experiments, leading to a proof of realization of $\mathcal{F}_{\text{X-SFE}}$ by the protocol $\Pi_{\text{X-SFE}}$.

Here is the complete code of Expt_0 (stated as it's overall experiment with $\mathcal{E}$).

Expt₀:

- (i) The challenger Chal gets from $\mathcal{E}$ the number of participants $\ell \leq m$ plus their (array of) inputs M . It computes and saves $y = f(M)$. Next, if $n_{\text{bad}} < n/3$ then it sets the input array M to all zero.
- (ii) The challenger Chal then runs the code $\mathcal{F}_{\text{X-Beaver}}$ as well as generates consistent input shares $\hat{a}_j = a_j + \rho_j, [a_j], \langle \rho_j \cdot \alpha \rangle^{\mathbb{E}}$ of each element a_j in the array M . It designates servers $\mathcal{C} = [1..n_{\text{bad}}]$ as corrupt and $[n_{\text{bad}} + 1..n]$ as honest. For each tuple generated in $\mathcal{F}_{\text{X-Beaver}}$ as well as input shares, it sends the first n_{bad} shares to $\mathcal{E}$. It also sends the first n_{bad} secret evaluation points θ_i (generated in code of $\mathcal{F}_{\text{X-Beaver}}$) to $\mathcal{E}$.
- (iii) Chal next emulates protocol Π for the honest servers with $\mathcal{E}$ running the part of the corrupt servers except for the following change at the end of emulation.
 - $n_{\text{bad}} < n/3$: In the last step where $l \cdot \alpha$ is opened, i.e. Chal sends shares of $l \cdot \alpha$ of the honest servers to $\mathcal{E}$, Chal re-calculates the shares of honest servers differently so that the opening will be to $l - y + f(\vec{0})$ instead. This is possible as described in detail above in the description of $\mathcal{S}$. Chal also outputs y to $\mathcal{E}$.
 - $n/2 > n_{\text{bad}} \geq n/3$: Chal outputs y to $\mathcal{E}$.

$n_{\text{bad}} \geq n/2$: At the end of the protocol, all honest servers output some value y_j at the end of the simulation. Chal also gets from $\mathcal{E}$ the values it is outputting for each corrupt server. If there is a value y' that has frequency $\geq n/2$, let $y^* = y'$. Else, let $y^* = \perp$. Chal outputs y^* to $\mathcal{E}$.

It is straightforward to inspect that the view of $\mathcal{E}$ in Expt_0 is identical to its view in its combined interaction with $\mathcal{F}_{\text{X-SFE}}$ and simulator $\mathcal{S}$, as Chal has just combined the code of $\mathcal{F}_{\text{X-SFE}}$ and $\mathcal{S}$.

Expt_1 :

This experiment differs from Expt_0 in step (i) in that the array M is not reset to zero for the $n_{\text{bad}} < n/3$ case, and, in step (iii), again for the $n_{\text{bad}} < n/3$ case, the opening of $l \cdot \alpha$ is not adjusted and done just as in real protocol Π .

Lemma B.2. [Privacy] *The experiments Expt_0 and Expt_1 are identically distributed for $\mathcal{E}$.*

Proof. First note that for a random $l \cdot \alpha$ (which is independent of $y (= f(M))$): the pairs $\langle f(\vec{0}) + l \cdot \alpha, l \cdot \alpha - y + f(\vec{0}) \rangle$ and $\langle y + l \cdot \alpha, l \cdot \alpha \rangle$ are identically distributed for any y . Since $< n/3$ shares of a (probabilistic) encoding $^d[x]$ of x , for $d \geq n/3$, do not reveal any information about x , the experiments Expt_0 and Expt_1 are distributed identically for $\mathcal{E}$ by Lemma B.1. Note that in both these experiments, in the $n_{\text{bad}} < n/3$ case, the game aborts if the honest-majority Byzantine consensus output is $\perp$.

Expt_2 : This experiment is the same as the real world, i.e. Chal runs the honest servers in $\Pi_{\text{ONLINE-X-SFE}}$ against $\mathcal{E}$.

The proof is complete by the following two lemmas B.3 and 7.2, which together show that the statistical distance between Expt_1 and Expt_2 is negligible.

Lemma B.3. [Robustness] *When $n_{\text{bad}} < n/3$, every honest server's input to the honest-majority Byzantine consensus in step viii of the attestation phase is $f(M)$ with overwhelming probability.*

Proof. To start with, by Lemma 3.1, since the number of honest servers is more than $2n/3$, honest servers will correctly decode x , α , and $l \cdot \alpha$ with overwhelming probability. Further, since only $< n/3$ servers are corrupt, an honest server's computation of A and B in attestation step ii and A 's eventual decoding in step iv will be correct with overwhelming probability. This follows since at every multiplication gate and the output wire, the values $a_l + l$ and $a_r + r$ obtained using OpenRobust will be correct. Therefore, every honest server's computation of each $\hat{a}$ and l will be correct, and it follows that, with overwhelming probability, every honest server's input to the honest-majority Byzantine consensus is $f(M)$.

Lemma 7.2 (repeated) [Correctness] *When $n_{\text{bad}} \in [n/3, n/2 - 1]$, if an honest server's input to the honest-majority Byzantine consensus in step viii of the attestation phase is not $\perp$, then with overwhelming probability it is $f(M)$.*

Proof. The proof of this lemma is in the main section.

Lemma 7.2 immediately implies that if the honest-majority Byzantine consensus in the attestation phase is a value $y \neq \perp$, then $y = f(M)$.

C Preprocessing Phase

In this section, we give an instantiation of the preprocessing phase, $\mathcal{F}_{\text{X-Beaver}}$, using [GSZ20]. Note that since this is preprocessing, the servers have ample time to run the protocol.⁸

C.1 Introduction to “Guaranteed Output Delivery Comes Free in Honest Majority MPC” [GSZ20]

Let c_I, c_M, c_R , and c_O be the number of input gates, multiplication gates, random gates, and output gates, respectively. We let $C = c_I + c_M + c_R + c_O$ be the size of the circuit. In our case, there are no private inputs (for our preprocessing phase), so $c_I = 0$. In [GSZ20], they only consider output gates that deliver output to all parties. In our application, we need another kind of output gate which opens the output to a single target party. However, this is easily handled as the output protocol in [GSZ20] anyway chooses one party P_{king} for accumulation, decoding, and dissemination, and so for our targeted delivery, we can just let P_{king} be that target party (without the dissemination part).

The MPC protocol in [GSZ20] achieves $(n/2, n/2, n/2)$ -security with total communication complexity $O(n \cdot \phi \cdot |C| + n^3 \lambda^2 + n^6 \cdot \lambda)$ bits plus $O(n^3 \cdot \lambda)$ bits on the assumed broadcast channel. Further, the proof of security of this dispute-control based protocol in [GSZ20] shows that if the protocol outputs all final outputs without failure, then *all internal gates were correctly computed in terms of sharings*. In other words, if the final output gates are output gates of a circuit C consisting of random gates, multiplication gates, and output gates, then if the protocol exits without failing, then the output of each random gate (in the rerun which caused no dispute) is indeed a t -sharing of a randomly distributed number in $\mathbb{E}$. Similarly, the output of a multiplication gate is indeed a t -sharing of ab , where the inputs are (inductively) correct t -sharings of a and b . This inductive property of their proof is not commonly highlighted, but we will use it in our complexity analysis.

We remark that addition gates and multiplication by public scalars do not require any communication as these can be instantiated by local computation of each party. Using this result, we show the communication complexity of realizing our preprocessing ideal functionality $\mathcal{F}_{\text{X-Beaver}}$ in the next subsection.

⁸We also remark that our targeted applications are multi-client distributed-server MPC. In these settings, the (VSS) input sharing by the clients can be unavoidably time consuming for reasons such as humans entering their inputs (e.g. in elections, auctions etc.) or clients that are not online in real-time. Thus, it is not unreasonable to assume that there is ample time for preprocessing.

C.2 Instantiation

We can express $\mathcal{F}_{\text{X-Beaver}}$ as a joint computation between n servers $S_1, \dots, S_n$, where S_i has as input θ_i , $\{w_i^{(k)}\}_{k \in [\ell]}$, and rand_i etc. However, since all these quantities are supposed to be random and independent (except for the requirement that some of these must be non-zero which happens with overwhelming probability as our field of operation is $\mathbb{E}$ with $|\mathbb{E}| > 2^\lambda$), it is best to generate these quantities using random gates and then output the quantity to a particular party using an output gate (see the remark in Section C.1).

Protocol CLONE

Given: A sharing $[s]_t$ employing public points of evaluation $\zeta_1, \dots, \zeta_n$.

Output: A sharing $[s]_{t'}$, possibly to smaller $t' \leq t$, and using the same evaluation points. Additionally, parties collectively get t -sharings of each party's share in this new $[s]_{t'}$.

1. Parties invoke $t' - 1$ random gates to get $[v_i]_t$, for $i \in [t' - 1]$ and each v_i random and independent in $\mathbb{E}$.
2. Parties locally compute (using public scalar multiplications) n t -sharings, $U_j = [s + \sum_{\ell=1}^{t'-1} \zeta_j^\ell \cdot v_\ell]_t = \Sigma[s]_t + \sum_{\ell=1}^{t'-1} \zeta_j^\ell \cdot [v_\ell]_t$, for $j \in [n]$.
3. For each $j \in [n]$, the parties use an output gate to open U_j to P_j , i.e. P_j gets $s + \sum_{\ell=1}^{t'-1} \zeta_j^\ell \cdot v_\ell$. Note that this collectively forms $[s]_{t'}$ using the same evaluation points. Also, note the parties already have (in Step 2) t -sharing of each party's share $s + \sum_{\ell=1}^{t'-1} \zeta_j^\ell \cdot v_\ell$. While these additional sharings are not using an output gate, the correct functioning of the MPC protocol's random gates implies that step 1 was performed correctly, and further the correctness of the output gate implies that these are indeed the sharings of each party's shares in new $[s]_{t'}$.

We now give the circuit to compute an extended and authenticated Beaver tuple

$$[l], [r], [lr], \langle l \cdot \alpha \rangle^\mathbb{E}, \langle r \cdot \alpha \rangle^\mathbb{E}, \langle lr \cdot \alpha \rangle^\mathbb{E}.$$

1. Assume $\langle \alpha \rangle$ has already been computed, as it is used in all tuples.
2. Using random gates compute $\langle l \rangle, \langle r \rangle$.
3. Using a multiplication gate compute $\langle lr \rangle$.
4. Using three multiplication gates, compute $\langle l \cdot \alpha \rangle, \langle r \cdot \alpha \rangle, \langle lr \cdot \alpha \rangle$.
5. Next, use the following circuit to convert a $\langle x \rangle$ to $\langle x \rangle^\mathbb{E}$ on $\langle l \cdot \alpha \rangle, \langle r \cdot \alpha \rangle, \langle lr \cdot \alpha \rangle$.
6. Clone $\langle l \rangle, \langle r \rangle, \langle lr \rangle$ to $[l]_{n/3}, [r]_{n/3}, [lr]_{n/3}$.
7. Output $[l]_{n/3}, [r]_{n/3}, [lr]_{n/3}$ and $\langle l \cdot \alpha \rangle^\mathbb{E}, \langle r \cdot \alpha \rangle^\mathbb{E}, \langle lr \cdot \alpha \rangle^\mathbb{E}$.

We now describe the circuit to convert $\langle x \rangle$ to $\langle x \rangle^\mathbb{E}$, along with outputting each party P_j 's secret authentication key θ_j to P_j . While the extended tuples needed in the attestation phase all have the same θ_j for P_j , the extended tuples needed for the input phase have random and independent $w_j^{(\ell)}$ for each input x_ℓ . So, we give this circuit as generating a fresh θ_j for each conversion.

1. The parties first run the cloning circuit to get a resharing of $\langle x \rangle$ to the same $t = n/2$. However, now they additionally got shares of each party's share in $\langle x \rangle$. We let $a^{(j)}$ denote this share of party P_j in $\langle x \rangle$. Thus, the parties also have $\langle a^{(j)} \rangle$ (for $j \in [n]$).
2. Use random gates to generate $\langle \theta_j \rangle$ for each $j \in [n]$.
3. Use random gates to generate $\langle b_{ij} \rangle$ for each $i, j \in [n]$.
4. For each $i, j \in [n]$, use a multiplication gate (and an addition gate) to compute $\langle a^{(i)} + b_{ij} * \theta_j \rangle$, and use an output gate to output this to P_j . Also, output b_{ij} to P_i .
5. For each j , output θ_j to P_j .

The total number of gates in the above conversion are: (1) $n/2$ random gates and n output gates in the cloning step, (2) n random gates in step 2, (3) n^2 random gates in step 3, (4) n^2 multiplication gates and $2n$ output gates in step 4, (5) $2n$ output gates in step 5. Since, the (amortized) communication cost of each gate is $O(n\phi)$, the total communication cost of this conversion is $O(n^3 \cdot \log |\mathbb{E}|)$. Since this conversion was the dominant cost to compute an extended and authenticated Beaver tuple, it follows that the amortized cost of computing such a tuple is $O(n^3 \cdot \log |\mathbb{E}|)$, and hence the total communication cost of the preprocessing protocol is $O(n^3 \cdot (|C| + L) \cdot \log |\mathbb{E}| + n^3 \lambda^2 + n^6 \lambda)$ bits plus $O(n^3 \cdot \lambda)$ bits in the assumed broadcast channel, where L is the number of (potential) clients.

D Concrete Example

In this section, we compare offline-online enhanced- $n/3$ protocol of ours against the honest-majority fully-secure protocol of [GSZ20]. The example below could be representative of US Treasury market auction (see <https://home.treasury.gov/services/treasury-auctions>).

For a circuit C , the total communication cost of the preprocessing protocol (when instantiated with [GSZ20]) is $O(n^3 \cdot (|C| + L) \cdot \log |\mathbb{E}| + n^3 \lambda^2 + n^6 \lambda)$ bits plus $O(n^3 \cdot \lambda)$ bits in the assumed broadcast channel, where L is the number of (potential) clients that provide inputs.

Since the constant in big-O notation of [GSZ20] is 7.5, the above complexity can be concretely calculated to be $44 \cdot n^3 \cdot |C| \cdot \log |\mathbb{E}|$ and $15 \cdot n^3 \cdot L \cdot \log |\mathbb{E}|$. As an example, if $n = 32 = 2^5$, and $|C| = 2^{16}$, and $L = 2^{20}$, i.e. a circuit with 64K multiplication gates and number of clients of similar order, then, with $\log |\mathbb{E}| = 2^7$, we get the total communication to be around $2^6 \cdot 2^{15} \cdot 2^{16} \cdot 2^7 = 2^{44}$, which is 16 terabits. On a 10 gigabits per second fiber-optic network, this large circuit needs preprocessing time of less than thirty minutes (even without adjusting for parallel communication of the n parties). If we divide this by n (as all n parties can send their messages in parallel), this amounts to one minute.

For the same setting, where the base field $\log |\mathbb{F}| = 16$, we estimate the communication of the online phase of our protocol to be approximately $2 \cdot n \cdot |C| \cdot \log |\mathbb{F}|$, which is $2 \cdot 2^5 \cdot 2^{16} \cdot 16 = 2^{26}$, which is 64 megabits. On the same 10 gigabits per

second fiber-optic network, this will take 6.4 milliseconds. Again if the n servers send their messages in parallel, that is less than a millisecond.

In contrast, the dominant term of [GSZ20] is the $O(n^6 \cdot \lambda)$ bits, which for the same setting can be estimated as $n^6 \cdot \lambda = 2^{30} \cdot 128 = 2^{37}$, which is 128 gigabits. This will take 12.8 seconds ignoring the additional required broadcast channel implementation (and about one-third of a second if parallel transmissions are accounted for). Note this is about 2000 times slower than the online phase of our protocol.